

Contents

1	Manual de Choose Your Destiny	4
1.1	Requerimientos e Instalación	8
1.1.1	Instalación en Windows	8
1.1.2	Instalación en Linux, BSDs y compatibles	8
1.2	CYDC (Compilador)	9
1.3	CYD Character Set Converter	12
1.4	Sintaxis Básica	12
1.4.1	Modo Colon Estricto (Enforced Colon Syntax)	14
1.4.2	Directiva INCLUDE (Proyectos multi-archivo)	15
1.5	Librerías incluidas	15
1.5.1	math16_32.cyd — aritmética de 16 y 32 bits	15
1.5.2	strings.cyd — cadenas de texto	16
1.6	Rutinas nativas (IMPORT / CALL)	17
1.6.1	Cómo escribir la rutina	17
1.6.2	Ejemplo	18
1.6.3	Escribir la rutina en el propio guion (ASM ... ENDASM)	18
1.6.4	Bloques con varias entradas (EXPORTS)	19
1.6.5	Acceder a los datos del motor	19
1.6.6	Llamar a otra rutina nativa (USES + CYD_CALL)	20
1.6.7	Llamar a servicios del motor (CYD_SYSCALL)	21
1.6.8	Rutinas no usadas	21
1.6.9	Targets soportados	22
1.7	Variables y expresiones numéricas	22
1.8	Control de flujo y expresiones condicionales	24
1.9	Asignaciones e indirección	26
1.10	Constantes	28
1.11	Arrays o “secuencias”	28
1.12	Datos inmutables (DATA / READ / RESTORE / DATAEND())	29
1.13	Constantes anchas (WORD / DWORD / cadenas)	30
1.14	Literales cómodos (carácter, rangos, repetición)	31
1.15	Listado de comandos	32
1.15.1	INCLUDE “ruta/al/archivo.cyd”	32
1.15.2	LABEL ID	32
1.15.3	#ID	32
1.15.4	DECLARE expression AS ID	32
1.15.5	CONST ID = expression	32
1.15.6	DIM ID(expression)	32
1.15.7	DIM ID(expression) = {expression, expression...}	32
1.15.8	DATA expression, expression...	33
1.15.9	READ varID	33
1.15.10	READ [varID]	33
1.15.11	RESTORE	33
1.15.12	RESTORE ID	33
1.15.13	DATAEND()	33
1.15.14	GOTO ID	33
1.15.15	GOSUB ID	33
1.15.16	RETURN	33
1.15.17	IF condexpression THEN ... ENDIF	33
1.15.18	IF condexpression THEN ... ELSE ... ENDIF	33
1.15.19	IF condexpression1 THEN ... ELSEIF condexpression2 THEN ... ELSE ... ENDIF	33

1.15.20 WHILE (condexpression) ... WEND	34
1.15.21 DO ... UNTIL (condexpression)	34
1.15.22 SELECT varexpression ... CASE valor[,valor...] ... [CASE ELSE ...] ENDSELECT	34
1.15.23 ENUM [nombre] { A, B=valor, C, ... }	34
1.15.24 SWAP varID, varID	34
1.15.25 FOR varID = varexpression TO varexpression [STEP expression] ... NEXT [varID]	34
1.15.26 SET varID TO varexpression	34
1.15.27 SET varID TO WORD expression / DWORD expression / "cadena"	34
1.15.28 SET [varID] TO varexpression	34
1.15.29 SET varID TO {varexpression1, varexpression2,...}	34
1.15.30 SET [varID] TO {varexpression1, varexpression2,...}	35
1.15.31 LET varID = varexpression	35
1.15.32 LET [varID] = varexpression	35
1.15.33 LET varID = {varexpression1, varexpression2,...}	35
1.15.34 LET [varID] = {varexpression1, varexpression2,...}	35
1.15.35 LET varID += varexpression	35
1.15.36 LET [varID] += varexpression	35
1.15.37 LET arrayID(varexpression) += varexpression	35
1.15.38 LET varID -= varexpression	35
1.15.39 LET [varID] -= varexpression	35
1.15.40 LET arrayID(varexpression) -= varexpression	35
1.15.41 END	35
1.15.42 CLEAR	35
1.15.43 CENTER	35
1.15.44 WAITKEY	36
1.15.45 OPTION GOTO ID	36
1.15.46 OPTION GOSUB ID	36
1.15.47 OPTION VALUE(varexpression) GOTO ID	36
1.15.48 OPTION VALUE(varexpression) GOSUB ID	36
1.15.49 CHOOSE	36
1.15.50 CHOOSE IF WAIT expression THEN GOTO ID	36
1.15.51 CHOOSE IF CHANGED THEN GOSUB ID	37
1.15.52 OPTIONSEL()	37
1.15.53 NUMOPTIONS()	37
1.15.54 OPTIONVAL()	37
1.15.55 CLEAROPTIONS	37
1.15.56 MENUCONFIG varexpression, varexpression, varexpression, varexpression	37
1.15.57 MENUCONFIG varexpression, varexpression, varexpression	37
1.15.58 MENUCONFIG varexpression, varexpression	37
1.15.59 CHAR varexpression	37
1.15.60 REPCHAR expression, expression	38
1.15.61 TAB expression	38
1.15.62 PRINT varexpression	38
1.15.63 PAGEPAUSE expression	38
1.15.64 INK varexpression	38
1.15.65 PAPER varexpression	38
1.15.66 BORDER varexpression	38
1.15.67 BRIGHT varexpression	38
1.15.68 FLASH varexpression	38
1.15.69 NEWLINE expression	38
1.15.70 NEWLINE	38
1.15.71 BACKSPACE expression	38

1.15.72 BACKSPACE	38
1.15.73 SFX varexpression	39
1.15.74 PICTURE varexpression	39
1.15.75 DISPLAY varexpression	39
1.15.76 BLIT varexpression, varexpression, varexpression, varexpression AT varexpression, varexpression	39
1.15.77 WAIT expression	39
1.15.78 PAUSE expression	39
1.15.79 TYPERATE expression	39
1.15.80 MARGINS expression, expression, expression, expression	39
1.15.81 FADEOUT expression, expression, expression, expression	40
1.15.82 AT varexpression, varexpression	40
1.15.83 FILLATTR varexpression, varexpression, varexpression, varexpression, varexpression	40
1.15.84 PUTATTR varexpression, varexpression AT varexpression, varexpression	40
1.15.85 PUTATTR varexpression AT varexpression, varexpression	40
1.15.86 GETATTR (varexpression, varexpression)	41
1.15.87 ATTRVAL (expression COMMA expression COMMA expression COMMA expression)	41
1.15.88 ATTRMASK (expression COMMA expression COMMA expression COMMA expression)	41
1.15.89 RANDOM(expression)	41
1.15.90 RANDOM()	41
1.15.91 RANDOM(expression, expression)	41
1.15.92 INKEY(expression)	42
1.15.93 INKEY()	42
1.15.94 KEMPSTON()	42
1.15.95 MIN(varexpression,varexpression)	42
1.15.96 MAX(varexpression,varexpression)	42
1.15.97 YPOS()	42
1.15.98 XPOS()	42
1.15.99 WINDOW expression	42
1.15.100 CHARSET expression	42
1.15.101 RANDOMIZE expression	43
1.15.102 RANDOMIZE	43
1.15.103 TRACK varexpression	43
1.15.104 PLAY varexpression	43
1.15.105 LOOP varexpression	43
1.15.106 RAMSAVE varID, expression	43
1.15.107 RAMSAVE varID	43
1.15.108 RAMSAVE	43
1.15.109 RAMLOAD varID, expression	43
1.15.110 RAMLOAD varID	44
1.15.111 RAMLOAD	44
1.15.112 SAVE varexpression, varId, expression	44
1.15.113 SAVE varexpression, varId	44
1.15.114 SAVE varexpression	44
1.15.115 LOAD varexpression	44
1.15.116 SAVERESULT()	45
1.15.117 SDISK()	45
1.15.118 LASTPOS(ID)	45
1.16 Imágenes	45
1.17 Efectos de sonido	46

1.18	Melodías Vortex Tracker	47
1.19	Melodías WyzTracker	47
1.20	Cómo generar una aventura	48
1.20.1	make_adventure.py (helper CLI)	49
1.20.2	Windows	49
1.20.3	Compilador GUI (make_adventure_gui v1.0.0)	51
1.20.4	Linux, BSDs	52
1.21	Ejemplos	53
1.21.1	Ejemplos Básicos	53
1.21.2	Ejemplos de Nivel Intermedio	55
1.21.3	Ejemplos Avanzados de Gráficos	56
1.21.4	Ejemplos de Proyectos Completos	57
1.21.5	Cómo usar los ejemplos	57
1.22	Juego de caracteres	58
1.23	Códigos de error	61
1.24	Referencias y agradecimientos	62
1.25	Licencias	62

1 Manual de Choose Your Destiny



Figure 1: logo

El programa es un compilador e intérprete para ejecutar historias de tipo “Escoge tu propia aventura” o aventuras por opciones y libro juegos, para el Spectrum 48, 128, +2 y +3.

Consiste una máquina virtual que va interpretando comandos que se encuentra durante el texto para realizar las distintas acciones interactivas y un compilador que se encarga de traducir la aventura desde un lenguaje similar a BASIC, con el que se escribe el guion de la aventura, a un fichero interpretable por el motor.

Además, también puede mostrar imágenes comprimidas y almacenadas en el mismo disco, así como efectos de sonido basados en BeepFX de Shiru, melodías tipo PT3 creadas con Vortex Tracker o melodías creadas con WyzTracker 2.0.

- [Manual de Choose Your Destiny](#)
 - [Requerimientos e Instalación](#)
 - * [Instalación en Windows](#)
 - * [Instalación en Linux, BSDs y compatibles](#)

- CYDC (Compilador)
- CYD Character Set Converter
- Sintaxis Básica
 - * Modo Colon Estricto (Enforced Colon Syntax)
 - * Directiva INCLUDE (Proyectos multi-archivo)
- Variables y expresiones numéricas
- Control de flujo y expresiones condicionales
- Asignaciones e indirección
- Constantes
- Arrays o “secuencias”
- Datos inmutables (DATA / READ / RESTORE / DATAEND())
- Constantes anchas (WORD / DWORD / cadenas)
- Literales cómodos (carácter, rangos, repetición)
- Listado de comandos
 - * INCLUDE “ruta/al/archivo.cyd”
 - * LABEL ID
 - * #ID
 - * DECLARE expression AS ID
 - * CONST ID = expression
 - * DIM ID(expression)
 - * DIM ID(expression) = {expression, expression...}
 - * GOTO ID
 - * GOSUB ID
 - * RETURN
 - * IF condexpression THEN ... ENDIF
 - * IF condexpression THEN ... ELSE ... ENDIF
 - * IF condexpression1 THEN ... ELSEIF condexpression2 THEN ... ELSE ... ENDIF
 - * WHILE (condexpression) ... WEND
 - * DO ... UNTIL (condexpression)
 - * SET varID TO varexpression
 - * SET [varID] TO varexpression
 - * SET varID TO {varexpression1, varexpression2,...}
 - * SET [varID] TO {varexpression1, varexpression2,...}
 - * LET varID = varexpression
 - * LET [varID] = varexpression
 - * LET varID = {varexpression1, varexpression2,...}
 - * LET [varID] = {varexpression1, varexpression2,...}
 - * LET varID += varexpression
 - * LET [varID] += varexpression
 - * LET arrayID(varexpression) += varexpression
 - * LET varID -= varexpression
 - * LET [varID] -= varexpression
 - * LET arrayID(varexpression) -= varexpression
 - * END
 - * CLEAR

- * CENTER
- * WAITKEY
- * OPTION GOTO ID
- * OPTION GOSUB ID
- * OPTION VALUE(varexpression) GOTO ID
- * OPTION VALUE(varexpression) GOSUB ID
- * CHOOSE
- * CHOOSE IF WAIT expression THEN GOTO ID
- * CHOOSE IF CHANGED THEN GOSUB ID
- * OPTIONSEL()
- * NUMOPTIONS()
- * OPTIONVAL()
- * CLEAROPTIONS
- * MENUCONFIG varexpression, varexpression, varexpression, varexpression
- * MENUCONFIG varexpression, varexpression, varexpression
- * MENUCONFIG varexpression, varexpression
- * CHAR varexpression
- * REPCHAR expression, expression
- * TAB expression
- * PRINT varexpression
- * PAGEPAUSE expression
- * INK varexpression
- * PAPER varexpression
- * BORDER varexpression
- * BRIGHT varexpression
- * FLASH varexpression
- * NEWLINE expression
- * NEWLINE
- * BACKSPACE expression
- * BACKSPACE
- * SFX varexpression
- * PICTURE varexpression
- * DISPLAY varexpression
- * BLIT varexpression, varexpression, varexpression, varexpression AT varexpression, varexpression
- * WAIT expression
- * PAUSE expression
- * TYPERATE expression
- * MARGINS expression, expression, expression, expression
- * FADEOUT expression, expression, expression, expression
- * AT varexpression, varexpression
- * FILLATTR varexpression, varexpression, varexpression, varexpression, varexpression
- * PUTATTR varexpression, varexpression AT varexpression, varexpression
- * PUTATTR varexpression AT varexpression, varexpression
- * GETATTR (varexpression, varexpression)

- * ATTRVAL (expression COMMA expression COMMA expression COMMA expression)
- * ATTRMASK (expression COMMA expression COMMA expression COMMA expression)
- * RANDOM(expression)
- * RANDOM()
- * RANDOM(expression, expression)
- * INKEY(expression)
- * INKEY()
- * KEMPSTON()
- * MIN(varexpression,varexpression)
- * MAX(varexpression,varexpression)
- * YPOS()
- * XPOS()
- * WINDOW expression
- * CHARSET expression
- * RANDOMIZE expression
- * RANDOMIZE
- * TRACK varexpression
- * PLAY varexpression
- * LOOP varexpression
- * RAMSAVE varID, expression
- * RAMSAVE varID
- * RAMSAVE
- * RAMLOAD varID, expression
- * RAMLOAD varID
- * RAMLOAD
- * SAVE varexpression, varId, expression
- * SAVE varexpression, varId
- * SAVE varexpression
- * LOAD varexpression
- * SAVERESULT()
- * ISDISK()
- * LASTPOS(ID)
- Imágenes
- Efectos de sonido
- Melodías Vortex Tracker
- Melodías WyzTracker
- Cómo generar una aventura
 - * make_adventure.py (helper CLI)
 - * Windows
 - * Compilador GUI (make_adventure_gui v1.0.0)
 - * Linux, BSDs
- Ejemplos
 - * Ejemplos Básicos

- `examples\test` - Introducción al motor
 - `examples\multicolumn_menu` - Menús en múltiples columnas
 - `examples\guess_the_number` - Juego de adivinanzas
 - `examples\input_test` - Entrada de teclado y arrays
 - `examples\windows` - Ventanas múltiples
 - * Ejemplos de Nivel Intermedio
 - `examples\ETPA_ejemplo` - Libro “Elige Tu Propia Aventura”
 - `examples\include_demo` - Organización multi-archivo
 - * Ejemplos Avanzados de Gráficos
 - `examples\blit` - Introducción a BLIT
 - `examples\blit_island` - Gráficos avanzados
 - `examples\Rocky_Horror_Show` - Animación de personajes
 - `examples\CYD_presents` - Efectos visuales complejos
 - `examples\Golden_Axe_select_character` - Selección dinámica con colores
 - * Ejemplos de Proyectos Completos
 - `examples\SCUMM_16` - Interfaz tipo SCUMM/LucasArts
 - `examples\Delerict` - Aventura completa con motor propio
 - * Cómo usar los ejemplos
 - Juego de caracteres
 - Códigos de error
 - Referencias y agradecimientos
 - Licencias
-

1.1 Requerimientos e Instalación

Estos son los requerimientos externos de la herramienta:

- Python 3.11 o superior
- SjAsmPlus 1.20.3 o superior
- Descompresor de ficheros ZIP

Si se actualiza una versión más antigua, es recomendable NO sobrescribirla. Es mejor renombrar el directorio de la versión antigua, descomprimir la nueva versión, copiar los archivos de tu aventura a la nueva versión y configurar de nuevo el guion `make_adv` para cada caso.

Estas son las instrucciones más detalladas para cada sistema operativo:

1.1.1 Instalación en Windows

La instalación es sencilla, simplemente descomprimir el fichero ZIP correspondiente descargado de la sección [Releases](#) del repositorio. En este caso, está todo incluido en el paquete. Se requiere Windows 10 o superior, versión de 64 bits (32 bits no soportados).

1.1.2 Instalación en Linux, BSDs y compatibles

Para estos sistemas, los requerimientos son:

- GNU/Linux / Unix / macOS / BSD con shell compatible con BASH.
- Python 3.11 o superior
- Todos los programas comunes del sistema (grep, cat, etc...).
- Todos los programas requeridos para compilar programas C en el sistema (libc, libstdc++, g++, GNU make, etc...).
- Autotools (autoconf, automake, aclocals...).
- wget
- git

Estos requerimientos son necesarios para compilar SjasPlus. No existe distribución en binario de esta herramienta para sistemas compatibles UNIX, con lo que se requiere compilarla directamente.

Debido a la heterodoxa naturaleza de las diferentes distribuciones, me resulta imposible dar instrucciones detalladas para instalar los requerimientos en cada caso en particular, con lo que se requieren conocimientos por parte del usuario para ello.

Lo primero que hay que hacer es clonar el repositorio del motor en cualquier lugar que creas conveniente y de forma recursiva para bajarse las dependencias:

```
git clone --recursive https://github.com/cronomantic/ChooseYourDestiny.git
cd ChooseYourDestiny
```

Ahora se puede proceder a compilar SjasPlus con los siguientes comandos:

```
cd external/sjasplus
make clean
make
cd ..
```

Y con esto ya tenemos preparadas las dependencias. Como último paso, debemos poner el script de compilación como ejecutable:

```
cd ../../
chmod a+x make_adv.sh
```

Y con esto ya podrías usar la herramienta en Linux.

1.2 CYDC (Compilador)

Este programa es el compilador que traduce el texto de la aventura a un fichero TAP o DSK. Además de compilar la aventura en un fichero interpretable por el motor, realiza una búsqueda de las mejores abreviaturas para reducir el tamaño del texto.

```
cydc_cli.py [-h] [-l MIN_LENGTH] [-L MAX_LENGTH] [-s SUPERSET_LIMIT]
            [-T EXPORT-TOKENS_FILE] [-t IMPORT-TOKENS-FILE] [-C EXPORT-CHARSET]
            [-c IMPORT-CHARSET] [-S] [-n NAME] [-img IMAGES_PATH] [-trk TRACKS_PATH]
            [-sfx SFX_ASM_FILE] [-scr LOAD_SCR_FILE] [-v] [-V] [-trim] [-dce] [-code]
            [--no-strict-colons] [--max-errors MAX_ERRORS] [-pause PAUSE_AFTER_LOAD]
            [-wyz] [-il NUM_IMAGE_LINES] [-720]
            {48k,128k,plus3,mld,mld128} input.cyd SJASPLUS_PATH OUTPUT_PATH
```

- **-h:** Muestra la ayuda

- **-l MIN_LENGTH**: La longitud mínima de las abreviaturas a buscar (por defecto, 3).
- **-L MAX_LENGTH**: La longitud máxima de las abreviaturas a buscar (por defecto, 30).
- **-s SUPERSET_LIMIT**: Límite para el superconjunto de la heurística de la búsqueda (por defecto, 100).
- **-T EXPORT-TOKENS_FILE**: Exportar al fichero JSON indicado por el parámetro las abreviaturas encontradas.
- **-t IMPORT-TOKENS-FILE**: Importar abreviaturas desde el fichero indicado y obviar la búsqueda de las mismas.
- **-C EXPORT-CHARSET**: Exporta el juego de caracteres 6x8 usado por defecto en formato JSON.
- **-c IMPORT-CHARSET**: Importa en formato JSON el juego de caracteres a emplear.
- **-S**: Si un fragmento de texto comprimido no cabe en un banco, se divide en dos entre el banco actual y el siguiente con esta opción activada. Si no, el fragmento pasa al banco siguiente.
- **-n NAME**: Nombre a usar para el fichero de salida (nombre para el fichero TAP o DSK), si no se define será el mismo que el fichero de entrada.
- **-scr LOAD_SCR_FILE**: Ruta hacia un fichero SCR con la pantalla de carga a usar.
- **-img IMAGES_PATH**: Ruta al directorio con las imágenes de la aventura.
- **-trk TRACKS_PATH**: Ruta al directorio con los ficheros PT3 de música AY o ficheros de WyzTracker.
- **-sfx SFX_ASM_FILE**: Ruta a un fichero ensamblador generado por BeepFx.
- **-v**: Modo verboso, da más información del proceso.
- **-V**: Indica la versión del programa.
- **-trim**: Elimina el código de aquellos comandos que no se usen en la aventura para reducir el tamaño del intérprete.
- **-dce**: Elimina el bytecode que nunca puede alcanzarse (por ejemplo, rutinas de una librería que incluyes pero no llamas). La alcanzabilidad sigue tanto los saltos como la ejecución secuencial (fall-through), así que no se quita nada que pudiera ejecutarse. Especialmente útil con las librerías incluidas.
- **-code**: Muestra el bytecode generado.
- **--no-strict-colons**: Permite sintaxis antigua sin separadores : entre sentencias en una misma línea.
- **--max-errors MAX_ERRORS**: Máximo de errores de parser/preprocesador que se informan antes de detenerse (por defecto 20).
- **-pause**: Número de segundos de pausa después de finalizar el proceso de carga, se puede cancelar con cualquier pulsación de tecla.
- **-wyz**: Usar música de tipo WyzTracker, en lugar de Vortex Tracker.
- **-il NUM_IMAGE_LINES**: Número de líneas que se emplearán en los ficheros de imagen (por defecto, 192).
- **-720**: En caso de usar el formato Plus3, se usará una imagen de disco de 720 Kb en lugar del tamaño estándar de 180 Kb.

-{48k,128k,plus3,mld,mld128}: Modelo de Spectrum a emplear: – **48k**: Versión para cinta en formato TAP, no incluye el reproductor de PT3 ni WyzTracker y se carga todo de una vez. Depende del tamaño de la memoria disponible. – **128k**: Versión para cinta en formato TAP, se carga todo de una vez en los bancos de memoria y depende del tamaño de la memoria disponible. – **plus3**: Esta versión generará un fichero DSK para ejecutarlo en Spectrum+3. Los recursos se

cargan dinámicamente según se necesiten y depende del tamaño en disco. – **mld**: Genera un fichero **.MLD** para cartucho **Dandanator Mini** en modo 48K. El texto, las imágenes y el bytecode se leen directamente desde los slots del cartucho (no usa bancos de RAM ni reproduce música), y aprovecha varios slots Dandanator (hasta ~256 KB de contenido). Usa el sistema de guardado del cartucho. – **mld128**: Genera un fichero **.MLD** para **Dandanator Mini** orientado a Spectrum 128K. Como el resto (texto/imágenes/bytecode) se lee desde los slots del cartucho, los bancos de RAM se reservan para lo que debe estar en RAM: la **música** (PT3/WyzTracker, que se reproduce desde RAM) y los **arrays DIM** (que deben ser escribibles; se reparten en bancos RAM dedicados que nunca colisionan con la música). Aprovecha muchos slots Dandanator (hasta ~480 KB de contenido). Limitación conocida: si la música, los arrays y el contenido fuerzan a usar todos los bancos de RAM, la compilación aborta con un error claro.

- **input.txt**: Fichero de entrada con el guion de la aventura.
- **SJASMPLUS_PATH**: Ruta al ejecutable de SjASMPlus.
- **OUTPUT_PATH**: Ruta donde se depositarán los ficheros de salida.

Nota sobre Dandanator (mld/mld128). El compilador genera un fichero **.MLD**, que es el juego, pero **no es directamente cargable**: hay que empaquetarlo en una **ROM de cartucho Dandanator Mini de 512 KB**. Para ello se usa la utilidad **mld2rom.py** incluida (Python puro; el firmware y el menú van incorporados, no hace falta ninguna ROM base externa):

```
python mld2rom.py -o mi_juego.rom -a mi_juego.MLD
```

El parámetro **-a** activa el autoarranque (lanza el juego sin pasar por el menú). La ROM resultante se puede grabar en un Dandanator Mini real o cargar en un emulador que lo soporte (p.ej. **EsSpectrum**, o **ZESarUX** con **--enable-dandanator --dandanator-rom mi_juego.rom**). La ROM generada es byte a byte idéntica a la de la herramienta oficial [dandanator-mini](#).

Como alternativa a la línea de comandos, la distribución incluye una utilidad gráfica independiente, **mld2rom_gui.py** (lanzadores **mld2rom_gui.cmd** / **mld2rom_gui.sh**), para crear la ROM con más opciones: nombre del juego, fuente del menú, gráfico de fondo, los textos del menú, autoarranque y efecto de borde.

El compilador es un programa escrito en Python, por lo que se requiere tener el entorno de Python instalado. Para mayor comodidad, se incluye en la distribución un Python embebido y un guion batch llamado **cydc.cmd** para lanzarlo desde la línea de comandos.

El compilador depende de un programa externo para funcionar, el ensamblador SjASMPlus, por lo que habrá que indicar en los parámetros de entrada la correspondiente ruta a este programa, el cual está incluido en la distribución en el directorio **tools**.

También hay que indicar un directorio de destino para dejar los ficheros resultantes de la compilación.

Opcionalmente, podemos indicar las rutas a directorios que contengan las imágenes comprimidas en formato **scr**, que se incluirán en la cinta o disco resultante. Lo mismo para los ficheros de tipo **PT3**. Ambos tipos de archivos deben estar nombrados con números de 3 dígitos, del 0 al 255, de tal forma que sean **000.scr**, **001.scr**, **002.PT3**, y así. Se indicarán los directorios que contienen ambos ficheros con los parámetros **-img** y **-trk** respectivamente.

Las imágenes serán comprimidas en un fichero de formato CSC. Las pantallas pueden ser completas, o se puede limitar el número de líneas horizontales para ahorrar memoria. Además detecta imágenes espejadas (simétricas) por el eje vertical, con lo que sólo almacena la mitad de la misma, pudiéndose incluso forzar este comportamiento y descartar el lado derecho de la imagen para ahorrar espacio. Más detalles de éste funcionamiento en la sección [Imágenes](#).

Para la música, podemos usar módulos de Vortex Tracker (PT3), o música creada con WyzTracker v2.0 y superiores. El comportamiento está también descrito en sus secciones correspondientes: [Melodías Vortex Tracker](#) y [Melodías WyzTracker](#).

Se pueden añadir efectos de sonido generados con la aplicación BeepFx de Shiru. Para utilizarlos, hay que exportar usando la opción **File->Compile** del menú superior. En la ventana flotante que aparece, debemos asegurarnos de que tenemos seleccionado **Assembly** e **Include Player Code**, el resto de las opciones son indiferentes. Luego guardar el fichero en algún punto accesible para indicar la ruta desde la línea de comandos con la opción **-sfx**.

1.3 CYD Character Set Converter

Esta utilidad permite convertir juegos de caracteres en formato **.chr**, **.ch8**, **.ch6** y **.ch4** en un fichero utilizable por el compilador en formato JSON. Estos formatos son editables con ZxPaintbrush.

```
cyd_chr_conv.py [-h] [-w WIDTH_LOW] [-W WIDTH_UP] [-v] [-V] charset.chr charset.json
```

Los parámetros que soporta:

- **-w**, **-width_low**: Ancho de los caracteres (1-8) del juego de caracteres inferior.
- **-W**, **-width_high**: Ancho de los caracteres (1-8) del juego de caracteres superior.
- **-h**, **-help**: Muestra la ayuda.
- **-v**, **-verbose**: Modo verboso.
- **-V**, **-version**: Versión del programa.
- **charset.chr**: Huevo de caracteres de entrada.
- **charset.json**: Fichero con el juego de caracteres para el compilador.

El ancho de los caracteres utilizado por defecto es 6 para el juego de caracteres inferior (desde primero al número 127) y 4 para el superior (desde el número 145 hasta el 256), pero se pueden cambiar estos valores con los parámetros **-w** y **-W** respectivamente. Indicar que los caracteres del 128 al 144 son especiales, ya que se usan para los cursores y siempre tendrán ancho 8, con lo que el tamaño de la fuente será ignorado en esos caracteres. Si se desea definir un ancho específico para cada carácter tendrás que editarlo en el fichero JSON de salida. Tienes más información en la sección [Juego de caracteres](#).

1.4 Sintaxis Básica

Los comandos para el intérprete se delimitan dentro de dos pares de corchetes, abiertos y cerrados respectivamente. Todo texto que aparezca fuera de esto, se considera “texto imprimible”, incluidos los espacios y saltos de línea, y se presentarán como tal por el intérprete. Los comandos se separan entre sí con saltos de línea o dos puntos si están en la misma línea.

Los comentarios dentro del código se delimitan con `/*` y `*/`, todo lo que haya en medio se considera un comentario.

Este es un ejemplo resumido y auto explicativo de la sintaxis:

```
Esto es texto [[ INK 6 ]] Esto es texto de nuevo pero amarillo
  Sigue siendo texto [[
    /* Esto es un comentario y lo siguiente son comandos */
    WAITKEY
    INK 7: PAPER 0
  ]]
  Esto vuelve a ser texto pero blanco, y ¡ojo con el salto de línea que lo precede!
```

El intérprete recorre el texto desde el principio, imprimiéndolo en pantalla si es “texto imprimible”. Cuando una palabra completa no cabe en lo que queda de la línea, la imprime en la línea siguiente. Y si no cabe en lo que queda de pantalla, se genera una espera y petición al usuario de que pulse la tecla de confirmación para borrar la sección de texto y seguir imprimiendo (este último comportamiento es opcional).

Cuando el intérprete detecta comandos, los ejecuta secuencialmente, a menos que encuentre saltos. Los comandos permiten introducir lógica programable dentro del texto para hacerlo dinámico y variado según ciertas condiciones. La más común y poderosa es la de solicitar escoger al jugador entre una serie de opciones (hasta un límite de 8 a la vez), y que puede elegir con las teclas P y Q o los cursores, y seleccionar con `SPACE`, M o `ENTER`. También soporta un joystick de tipo Kempston para desplazarse por el menú. De nuevo, éste es un ejemplo auto explicativo:

```
[[ /* Comandos que ponen colores de pantalla y la borra */
  PAPER 0 /* Color de fondo a cero (negro)*/
  INK 7 /* Color de la tinta (blanco) */
  BORDER 0 /* Color del borde (negro) */
  CLEAR /* Borrar el área de texto (pantalla completa) */
  LABEL Localidad1]]Estás en la localidad 1. ¿Donde quieres ir?
[[OPTION GOTO Localidad2]]Ir a la localidad 2
[[OPTION GOTO Localidad3]]Ir a la localidad 3
[[CHOOSE]]
[[ LABEL Localidad2]]¡¡¡Lo lograste!!!
[[ GOTO Final]]
[[ LABEL Localidad3]]¡¡¡Estas muerto!!!
[[ GOTO Final]]
[[ LABEL Final : WAITKEY: END ]]
```

El comando `OPTION GOTO etiqueta` generará un punto de selección en el lugar en donde se haya llegado al comando. Cuando llegue al comando `CHOOSE`, el intérprete permitirá elegir al usuario entre uno de los puntos de opción que haya acumulados en pantalla hasta el momento. Se permiten un máximo de 32.

Al escoger una opción, el intérprete saltará a la sección del texto donde se encuentre la etiqueta correspondiente indicada en la opción. Las etiquetas se declaran con el pseudo-comando `LABEL identificador` dentro del código, y cuando se indica un salto a la misma, el intérprete comenzará a procesar a partir del punto en donde hemos declarado la etiqueta. En el caso del ejemplo, si elegimos

la opción 1, el intérprete saltará al punto indicado en `LABEL localidad2`, con lo que imprimirá el texto “*¡¡¡Lo lograste!!!*”, y después pasa a `GOTO Final` que hará un salto incondicional a donde está definido `LABEL Final` e ignorando todo lo que haya entre medias.

Los identificadores de las etiquetas sólo soportan caracteres alfanuméricos (cifras y letras), deben empezar con una letra y son sensibles al caso (se distinguen mayúsculas y minúsculas), es decir `LABEL Etiqueta` no es lo mismo que `LABEL etiqueta`. Los comandos, por el contrario, no son sensibles al caso, pero por claridad, es recomendable ponerlos en mayúsculas.

También se permite una versión acortada de las etiquetas precediendo el carácter `#` al identificador de la etiqueta, de tal manera que `#MiEtiqueta` es lo mismo que `LABEL MiEtiqueta`. Con esto, podríamos reescribir el anterior código así:

```
[[ /* Comandos que ponen colores de pantalla y la borra */
  PAPER 0 /* Color de fondo a cero (negro)*/
  INK 7 /* Color de la tinta (blanco) */
  BORDER 0 /* Color del borde (negro) */
  CLEAR /* Borrar el área de texto (pantalla completa) */
  LABEL Localidad1]]Estás en la localidad 1. ¿Donde quieres ir?
[[OPTION GOTO Localidad2]]Ir a la localidad 2
[[OPTION GOTO Localidad3]]Ir a la localidad 3
[[CHOOSE]]
[[ #Localidad2 ]]]¡¡¡Lo lograste!!!
[[ GOTO Final ]]
[[ #Localidad3 ]]]¡¡¡Estas muerto!!!
[[ GOTO Final]]
[[ #Final : WAITKEY: END ]]
```

Los comandos disponibles están descritos en su sección correspondiente.

1.4.1 Modo Colon Estricto (Enforced Colon Syntax)

A partir de la versión 1.2.x, el compilador aplica **Modo Colon Estricto por defecto**. Esto significa que cuando hay múltiples declaraciones de código en la misma línea dentro de bloques `[[ ]]`, **deben** estar separadas por dos puntos (`:`)

Sintaxis correcta (Modo Colon Estricto habilitado - POR DEFECTO):

```
[[ PRINT "Hola" : INK 5 : GOTO Label1 ]]
[[ SET mivar TO 10 : WAITKEY : END ]]
```

Sintaxis incorrecta (faltan dos puntos):

```
[[ PRINT "Hola" INK 5 GOTO Label1 ]] <-- ERROR: Las declaraciones deben estar separadas por dos puntos
```

Si necesitas soportar código antiguo sin separadores de dos puntos, pasa el parámetro `--no-strict-colons` al compilador:

```
cydc_cli.py --no-strict-colons 48k entrada.cyd sjasmplus salida
```

Puntos clave: - Los saltos de línea separan automáticamente las declaraciones, así que los dos puntos solo son necesarios cuando hay múltiples comandos en la misma línea - Los dos puntos son opcionales con el parámetro `--no-strict-colons` para compatibilidad retroactiva - Este cambio

mejora la legibilidad del código y previene ambigüedades en el análisis

1.4.2 Directiva INCLUDE (Proyectos multi-archivo)

El compilador permite dividir aventuras grandes en varios archivos fuente usando **INCLUDE**.

Sintaxis:

```
[[ INCLUDE "capitulos/capitulo1.cyd" ]]
```

Reglas y comportamiento: - **INCLUDE** se procesa en tiempo de compilación por el preprocesador. - La directiva debe estar dentro de bloques de código `[[ ]]`. - Las rutas relativas se resuelven respecto al archivo que contiene la directiva. - Se admiten inclusiones anidadas hasta 20 niveles. - Las inclusiones circulares se detectan y se reportan como error. - Un archivo incluido puede incluir otros archivos.

1.5 Librerías incluidas

El proyecto trae en el directorio `lib/` un conjunto de **librerías escritas en el propio lenguaje CYD** (no modifican la máquina virtual ni añaden dependencias). Son colecciones de subrutinas que se incorporan a tu programa con **INCLUDE** y se invocan con **GOSUB**. Cada librería se **auto-salta** sus rutinas (un **GOTO** interno al principio), así que puedes incluirla al comienzo de tu guion sin que el flujo de ejecución caiga dentro de las subrutinas: solo se ejecutan cuando las llamas con **GOSUB**.

```
[[
    INCLUDE "../lib/math16_32.cyd"
    INCLUDE "../lib/strings.cyd"
    ... tu programa ...
]]
```

Cada librería reserva un bloque de variables como espacio de trabajo. Los bloques **no se solapan**, de modo que puedes usar varias a la vez:

Librería	Variables reservadas
<code>math16_32.cyd</code>	224..254
<code>strings.cyd</code>	216..223

1.5.1 `math16_32.cyd` — aritmética de 16 y 32 bits

Da enteros anchos **sin signo** (16 bits: 0..65.535; 32 bits: 0..4.294.967.295) con multiplicación y división, que con las variables de 8 bits de CYD no eran viables por desbordar. Los operandos se cargan en unos «registros» (grupos de 2 ó 4 variables consecutivas, byte bajo primero): `A = m1A0..m1A3`, `B = m1B0..m1B3`, `C = m1C0..m1C3`. Las operaciones de 16 bits usan los 2 bytes bajos de cada uno.

Como los literales de CYD son de 8 bits, los valores anchos se cargan byte a byte con la asignación múltiple: `SET m1A0 TO {226, 4}` carga 1250 (= 0x04E2, byte bajo primero).

Rutina	Efecto
add16 / add32	A = A + B (envuelve al desbordar)
sub16 / sub32	A = A - B (envuelve; usa cmp para comprobar antes)
cmp16 / cmp32	mlCmp = 0/1/2 → A<B / A=B / A>B
shl16 / shl32	A = A << 1
shr16 / shr32	A = A >> 1
mul32	C = A * B (trunca a 32 bits)
mul1632	C(32) = A(16) * B(16) — nunca desborda (recomendada para puntuaciones)
div32	A = A / B (cociente), C = A mod B (resto)
print16 / print32	imprime A en decimal (destruye A)

Para dividir entre un valor de 16 bits, cárgalo en B con mlB2 = mlB3 = 0 y usa div32 (el cociente puede ser de 32 bits). El tier de 16 bits es ~2–4× más rápido; evita mul/div dentro de bucles muy apretados. Hay un ejemplo completo en `examples/math_library`.

Ejemplo: puntuación = enemigos × puntos + bonus = 1250 × 800 + 234567.

```
[[
  INCLUDE "../lib/math16_32.cyd"
  SET mlA0 TO {226, 4}      /* A = 1250 */
  SET mlB0 TO {32, 3}       /* B = 800 */
  GOSUB mul1632             /* C = 1000000 (sin desbordar) */
  SET mlA0 TO {@mlC0, @mlC1, @mlC2, @mlC3} /* A = C */
  SET mlB0 TO {71, 148, 3, 0} /* B = 234567 */
  GOSUB add32               /* A = 1234567 */
]]Puntuacion: [[ GOSUB print32 ]]
```

1.5.2 strings.cyd — cadenas de texto

Entrada por teclado, impresión y manejo de cadenas de caracteres guardadas en variables consecutivas (usando la indirección [`@ptr`]), terminadas en 0. Antes de llamar, fija los registros: `stBase` (índice de la primera variable del buffer), `stLen` (capacidad en caracteres) y, para copiar o comparar, `stB2` (índice del segundo buffer).

Rutina	Efecto
strClear	pone a 0 el buffer
strLen	cuenta caracteres hasta el 0 o el final → <code>stRes</code>
strPrint	imprime el buffer (CHAR) hasta el 0 o el final
strInput	lee una cadena del teclado (cursor, ENTER termina, DELETE borra)
strCopy	copia el buffer <code>stBase</code> → <code>stB2</code> (<code>stLen</code> bytes)
strCmp	compara <code>stBase</code> con <code>stB2</code> → <code>stRes</code> = 0/1/2 (menor/igual/mayor)

Ejemplo: pedir un nombre y saludar.


```
[[
    INCLUDE "../../lib/strings.cyd"
    SET stBase T0 0 : SET stLen T0 16
    GOSUB strInput
]]Hola, [[ GOSUB strPrint ]]
```

Hay un ejemplo completo en `examples/strings_library`.

1.6 Rutinas nativas (IMPORT / CALL)

Para tareas que la máquina virtual no puede hacer —leer o escribir un puerto hardware, tocar una dirección de memoria arbitraria, o un cálculo que necesita ir rápido— puedes escribir una pequeña rutina en **ensamblador Z80** y llamarla desde tu guion.

Avanzado, bajo tu propia responsabilidad. Esto ejecuta código nativo tuyo sin ninguna red de seguridad: un fallo en la rutina puede colgar o bloquear la máquina, igual que el código de BeepFX o WyzTracker en el que ya confías. Si no te manejas con ensamblador Z80, no necesitas esta función.

Intervienen dos palabras clave:

```
[[
    IMPORT beeper FROM "rutinas/beeper.asm"    ; declara (en compilación)
    ...
    CALL beeper                                ; la ejecuta (en tiempo de ejecución)
]]
```

- **IMPORT nombre FROM "fichero.asm"** registra una rutina nativa bajo nombre. Es una declaración de compilación (como `DECLARE` o `CONST`, **no** como `INCLUDE`, que incluye fuente CYD). Una ruta relativa se resuelve respecto a la carpeta de tu guion `.cyd`.
- **CALL nombre** ejecuta la rutina. Piénsalo como el equivalente nativo de `GOSUB`: `GOSUB etiqueta` llama a una subrutina escrita en CYD, `CALL nombre` llama a una escrita en Z80.

1.6.1 Cómo escribir la rutina

Escribes **solo el cuerpo** de la rutina — sin `ORG`, sin directivas, sin `SAVEBIN`. El compilador la enmarca, la ensambla **de forma aislada** (así un error en tu fichero da un fallo limpio y atribuible a *tu* fichero, sin romper la compilación del motor) y la coloca en memoria por ti, re-ensamblándola en su dirección final para que las direcciones absolutas funcionen sin más.

El contrato (ABI) es deliberadamente pequeño:

- La rutina entra con **DE = FLAGS**, la dirección base del array de 256 variables. Cada variable CYD `n` es el byte en `FLAGS + n`.
- **Las entradas y salidas viajan por variables.** El guion pone los argumentos en variables con `SET`, llama a la rutina y lee los resultados con `@var`. La rutina los lee y escribe en `FLAGS + n`. No hay otra convención de llamada que recordar.
- Debe terminar con **RET** y no debe saltar por su cuenta a rutinas internas del motor. Puede usar libremente `AF/BC/DE/HL/IX/IY` (el motor guarda y restaura `IX/IY`, que usa internamente). Para

lo que sí necesitas —acceder a los arrays entre bancos o llamar a otra rutina nativa— hay **servicios residentes** documentados más abajo (CYD_PEEK/CYD_POKE, CYD_ARR_MAP/CYD_ARR_FLUSH, CYD_CALL).

- **Debe caber en un banco de 16 KB** y no debe dejar el hardware en un estado que rompa el motor al volver (si cambia la paginación por \$7FFD, debe restaurarla).

1.6.2 Ejemplo

```
[[
  IMPORT peek FROM "peek.asm"
  DECLARE 0 as addr_lo
  DECLARE 1 as addr_hi
  DECLARE 2 as result
  SET addr_lo T0 0 : SET addr_hi T0 0    ; dirección 0000h
  CALL peek                                ; result <- byte en esa dirección
]]
```

con peek.asm (escribes solo esto):

```
ld a, (de)      ; DE = FLAGS -> A = FLAGS+0 = byte bajo de la dirección
ld l, a
inc de
ld a, (de)      ; A = FLAGS+1 = byte alto de la dirección
ld h, a         ; HL = la dirección a leer
inc de         ; DE -> FLAGS+2
ld a, (hl)      ; A = el byte en esa dirección
ld (de), a      ; FLAGS+2 = result
ret
```

Hay un ejemplo completo y ejecutable en `examples/import_demo`.

1.6.3 Escribir la rutina en el propio guion (ASM ... ENDASM)

No siempre quieres un fichero aparte. Puedes escribir el cuerpo **directamente en el .cyd** entre ASM nombre y ENDASM, y llamarlo igual con CALL:

```
[[
  ASM poner42
    ld a, 42
    ld (de), a      ; DE = FLAGS -> FLAGS+0 = 42
    ret
  ENDASM
  CALL poner42
]]
```

- El cuerpo se escribe **tal cual** (verbatim): comentarios ;, etiquetas, números 0x..... todo se pasa a sjasmpplus sin que CYD lo interprete. Puedes **sangrar el bloque** como quieras para alinearlos con el [[]]: CYD lleva las líneas de etiqueta (y de EQU) a la columna 0 por ti; basta con que las instrucciones queden más sangradas que las etiquetas.
- Vale la **misma ABI** que con IMPORT (entra con DE = FLAGS, termina en RET) y el mismo CALL.

ASM ... ENDASM no es más que un **IMPORT** con el cuerpo en línea. Si sjasmplus da un error, CYD te lo remapea a la **línea de tu .cyd**.

1.6.4 Bloques con varias entradas (EXPORTS)

Un bloque puede exponer **varias rutinas** que comparten ayudantes privados y se llaman entre sí con un **call** normal (están en el mismo banco). Se declaran con **EXPORTS**:

```
[[
  ASM matriz EXPORTS poner_a, poner_b
  poner_a:
    ld a, 42
    jr guardar
  poner_b:
    inc de
    ld a, 99
  guardar:
    ld (de), a      ; helper privado, compartido
    ret
  ENDASM
  CALL poner_a      ; FLAGS+0 = 42
  CALL poner_b      ; FLAGS+1 = 99
]]
```

Cada nombre de **EXPORTS** es un destino de **CALL**. El bloque se ensambla y coloca como **una sola unidad** (la unidad natural de una “librería”).

1.6.5 Acceder a los datos del motor

Al ensamblar tu rutina, CYD le inyecta por nombre las direcciones de las estructuras residentes, para que no tengas que adivinarlas:

Símbolo	Qué es
FLAGS	base del array de 256 variables (lo mismo que DE a la entrada)
SCREEN_BUFFER_PXL / SCREEN_BUFFER_ATT	buffer de imagen del motor (píxeles / atributos)
VIDEO_PXL / VIDEO_ATT	pantalla física del Spectrum (\$4000 / \$5800)

Por ejemplo, `ld hl, SCREEN_BUFFER_PXL` te da el buffer de imagen sin más.

1.6.5.1 Arrays Los arrays de tu guion (**DIM**) también son accesibles. Por cada array **<nombre>** CYD inyecta:

Símbolo	Qué es
ARR_<nombre>	dirección del elemento 0
ARR_<nombre>_LEN	número de elementos
ARR_<nombre>_BANK	banco físico donde vive, o \$FF si es residente

En 128K/+3 un array puede vivir en un **banco paginado** que tu rutina no puede mapear sin auto-expulsarse. Por eso el acceso va por **servicios residentes** que hacen la paginación por ti (en 48K, o si el banco es \$FF, son acceso directo). Todos preservan BC/DE/HL/IX/IY (salvo el resultado):

- **CYD_PEEK** — lee un byte. Entra A = banco, HL = dirección; sale A = byte.
- **CYD_POKE** — escribe un byte. Entra A = banco, HL = dirección, E = byte.
- **CYD_ARR_MAP** — copia el array entero a un buffer residente y te devuelve un puntero directo para trabajar sin paginar. Entra A = banco, HL = ARR_<n>, BC = ARR_<n>_LEN; sale HL = puntero de trabajo.
- **CYD_ARR_FLUSH** — vuelca ese buffer de vuelta al array. Mismos parámetros de entrada. (Solo un array mapeado a la vez.)

Ejemplo — incrementar el elemento 3 de inv:

```
[[
    DIM inv(4) = {10, 20, 30, 40}
    ASM inc3
        ld a, ARR_inv_BANK
        ld hl, ARR_inv + 3
        call CYD_PEEK      ; A = inv[3]
        inc a
        ld e, a
        ld a, ARR_inv_BANK
        ld hl, ARR_inv + 3
        call CYD_POKE      ; inv[3] = inv[3] + 1
        ret
    ENDASM
    CALL inc3
]]
```

Si vas a tocar muchos elementos, CYD_ARR_MAP + CYD_ARR_FLUSH es más rápido: mapeas una vez, recorres el puntero a pelo, y vuelcas al final.

1.6.6 Llamar a otra rutina nativa (USES + CYD_CALL)

Dos rutinas del **mismo bloque** (EXPORTS) se llaman con un `call` normal. Para llamar a una rutina de **otro bloque** (que en 128K/+3 puede estar en otro banco) usa el trampolín residente CYD_CALL y declara a quién llamas con USES:

```
[[
    ASM saludar EXPORTS saluda
    saluda:
        ld a, 55
        ld (FLAGS+1), a
        ret
    ENDASM
    ASM principal USES saluda
        ld a, 11
        ld (FLAGS), a
]]
```

```

        ld a, RT_saluda      ; índice inyectado por USES
        call CYD_CALL       ; llama a 'saluda', esté en el banco que esté
        ret
    ENDASM
    CALL principal          ; FLAGS = [11, 55]
]]

```

- USES a, b, ... declara los callees; por cada uno CYD inyecta un índice RT_<nombre>.
- ld a, RT_<nombre> : call CYD_CALL ejecuta ese callee. Entra con DE = FLAGS (igual que un CALL del guion) y CYD hace la paginación por ti.

1.6.7 Llamar a servicios del motor (CYD_SYSCALL)

Algunas cosas las hace el motor, no la máquina virtual: **imprimir un carácter** o **leer el teclado**. Tu rutina puede pedírselas por un único punto de entrada, CYD_SYSCALL, indicando en A el **número de servicio** (constantes SVC_* que CYD inyecta) y los argumentos en registros:

```

[[
    ASM saluda
        ld e, 72            ; código de 'H'
        ld a, SVC_PRINT_CHAR
        call CYD_SYSCALL    ; imprime 'H' en el cursor
        ret
    ENDASM
    CALL saluda
]]

```

Servicios disponibles (empezamos con un conjunto pequeño; crecerá):

Servicio	Entrada / salida
SVC_PRINT_CHAR	E = carácter — lo imprime en el cursor (avanza, respeta tinta/papel/ventana)
SVC_WAIT_KEY	espera una tecla → A = código
SVC_INKEY	lee una tecla sin esperar → A = código (0 si ninguna)

Los números de servicio son un **contrato estable**: aunque el intérprete cambie por dentro, tus rutinas siguen valiendo sin recompilar. Trata CYD_SYSCALL como cualquier call (puede alterar los registros); en particular, no toques IY antes de llamarlo.

1.6.8 Rutinas no usadas

Una rutina IMPORT/ASM que **nadie llama** no se ensambla ni ocupa memoria: CYD la descarta sola. Puedes tener así una “librería” de rutinas y pagar solo por las que uses de verdad. (Los servicios residentes de arriba —broker de arrays, CYD_CALL, CYD_SYSCALL— también se eliminan del motor si ninguna rutina los usa.)

Hay un ejemplo con ASM, EXPORTS, arrays y CYD_CALL en `examples/inline_asm`.

1.6.9 Targets soportados

Las rutinas nativas funcionan en **48K**, **128K**, **+3** y **mld128**. En 128K, +3 y mld128 la rutina se coloca en un banco de RAM paginado y el motor lo pagina automáticamente alrededor del **CALL**. (El target **mld** estricto de 48K —un solo banco— no las soporta y la compilación aborta con un error claro.)

1.7 Variables y expresiones numéricas

Los valores numéricos constantes se puede expresar en base 10 por defecto, en hexadecimal con el prefijo **0x** o en binario con el prefijo **0b**. Por ejemplo, **240** sería en decimal, **0xF0** en hexadecimal y **0b11110000** en binario.

Hay a disposición del programador 256 contenedores de un byte (de 0 a 255) para almacenar valores, realizar operaciones y comparaciones con ellos. Constituyen el estado del programa. A partir de este momento, pueden ser llamados variables, banderas o “variables” indistintamente a lo largo de este documento.

Para referirnos a una variable en una expresión numérica, el número debe estar precedido por el carácter **@**. Pero es posible dar un nombre significativo a las variables usando el comando **DECLARE** de la siguiente manera:

```
[[
DECLARE 7 AS ColorTinta
INK @ColorTinta
]]
```

Hay que indicar que no se puede declarar una variable dos veces, ni puede haber etiquetas y variables con los mismos nombres, pero se permiten sinónimos de la siguiente forma:

```
[[
DECLARE 10 AS UnNombre
DECLARE 10 AS OtroNombre
]]
```

Así, tanto *UnNombre* como *OtroNombre* servirán para identificar el variable 10. Ten en cuenta de a pesar de tener distinto nombre, son la misma variable.

Para asignar valores a un variable, usamos el comando **SET varId T0 varexpression**, de la siguiente manera:

```
[[
SET 0 T0 1                /* Ponemos el variable cero a 1 */
SET variable T0 2         /* Ponemos el variable llamado variable a 2 */
SET variable2 T0 @variable + 2 /* Ponemos el variable llamado variable2 al dos sumado al valor del v
SET variable2 T0 @variable2 - (@variable + 2) /* Permite paréntesis */
]]
```

También tenemos ahora el siguiente sinónimo con **LET varId = varexpression**, de tal forma que los ejemplos anteriores se podrían reescribir así:

```
[[
  LET 0 = 1 /* Ponemos el variable cero a 1 */
  LET variable = 2 /* Ponemos el variable llamado variable a 2 */
  LET variable2 = @variable + 2 /* Ponemos el variable llamado variable2 al dos sumado al valor del va
  LET variable2 = @variable2 - (@variable + 2) /* Permite paréntesis */
]]
```

Como se puede ver, a un variable se le puede asignar el valor de una expresión matemática compuesta por números, funciones y variables. Como ya se ha indicado, las variables dentro de una expresión numérica, es decir, el lado derecho de una asignación **siempre deben estar precedidas del carácter @**.

Los operandos disponibles son:

- Suma: SET variable T0 @variable + 2
- Resta: SET variable T0 @variable - 2
- “AND” binario: SET variable T0 @variable & 2
- “OR” binario: SET variable T0 @variable | 2
- “NOT” binario o complemento de bits: SET variable T0 !@variable
- Desplazamiento de bits a la izquierda: SET variable T0 @variable << 2
- Desplazamiento de bits a la derecha: SET variable T0 @variable >> 2

Además, LET soporta operadores de atajo para modificar una variable en el mismo sitio:

- Incremento: LET variable += 2 (equivalente a LET variable = @variable + 2)
- Decremento: LET variable -= 2 (equivalente a LET variable = @variable - 2)

Estos también funcionan con indirección y arrays: LET [variable] += 1 y LET miArray(0) += 1.

El resultado de una expresión numérica no pueden ser mayor de 255 (1 byte) ni menor que cero (no se soportan números negativos). Si al realizar las operaciones se rebasan ambos límites, el resultado se ajustará al límite correspondiente, es decir, si una suma supera 255, se ajustará a 255 y una resta que dé un resultado inferior a cero, se quedará en cero.

Una cosa a destacar es que los operadores binarios **no son los mismos que los operadores lógicos de las expresiones condicionales** descritos más abajo. Un & no es lo mismo que un AND. Los operadores binarios realizan las correspondientes operaciones sobre los bits del variable.

La mayor parte de los comandos aceptan expresiones numéricas en sus parámetros, por ejemplo:

```
[[
INK 7
INK @7
]]
```

El primer comando pondrá el color del texto en blanco (color 7), mientras que el segundo pondrá el color del texto con el valor contenido en la variable número 7. Combinando con todo lo anteriormente descrito, se podría hacer:

```
[[
INK 3+4
```

```
INK @Var-1
11
```

Consulta la referencia de comandos para saber cuáles de ellos los aceptan.

Por último, hay una serie de comandos especiales llamados **funciones**. Estos comandos no pueden usarse por sí solos, sino que deben usarse dentro de una expresión numérica, ya que devuelven un valor a emplear dentro de ella.

- **RANDOM(expression)**: Devuelve un número aleatorio entre 0 y el valor indicado en **expression**.
- **RANDOM()** : Devuelve un número aleatorio entre 0 y 255. Es el equivalente a **RANDOM(255)**.
- **RANDOM(expression,expression)**: Devuelve un número aleatorio entre el valor indicado en el primer parámetro y el segundo, ambos inclusive.
- **INKEY()** Se espera hasta que se pulse una tecla y devuelve el código de la tecla pulsada (sólo teclado).
- **KEMPSTON()** Devuelve los botones pulsados en un joystick kempston conectado al Spectrum.
- **MIN(varexpression,varexpression)**: Acota el valor del primer parámetro al mínimo indicado por el segundo. Es decir, si el parámetro 1 es menor que el parámetro 2, se devuelve el parámetro 2, en otro caso se devuelve el 1.
- **MAX(varexpression,varexpression)**: Acota el valor del primer parámetro al máximo indicado por el segundo. Es decir, si el parámetro 1 es mayor que el parámetro 2, se devuelve el parámetro 2, en otro caso se devuelve el 1.
- **POSX()**: Devuelve la fila actual en la que se encuentra el cursor en coordenadas 8x8.
- **POSY()**: Devuelve la columna actual en la que se encuentra el cursor en coordenadas 8x8 (Debido a la naturaleza de la fuente de ancho variable, el valor devuelto será la columna 8x8 donde esté actualmente el cursor).
- **LASTPOS(ID)**: Devuelve el índice de la última posición del array dado.

Así que, por ejemplo **SET variable_no TO RANDOM(6)**, asigna a la variable *variable_no* un número aleatorio del 0 al 6. Y de la misma manera, podemos operar con ellos: **SET variable_no TO RANDOM(6) + @var + 1**

1.8 Control de flujo y expresiones condicionales

Para controlar el flujo de ejecución, tenemos diferentes comandos, que describiré ahora:

- **GOTO ID**

Salta a la etiqueta *ID*.

- **GOSUB ID**

Salto de subrutina, hace un salto a la etiqueta *ID*, pero devuelve la ejecución a la siguiente instrucción al **GOSUB** en cuanto encuentre un comando **RETURN**.

- **RETURN**

Retorna al punto posterior de la llamada de la subrutina, ver **GOSUB**

- **IF condexpression THEN ... ENDIF**

Si la expresión condicional *condexpression* resulta cierta, se imprime el texto y se ejecutan los comandos que haya desde **THEN** hasta **ENDIF**.

- **IF condexpression THEN ... ELSE ... ENDIF**

Si la expresión condicional *condexpression* resulta cierta, se imprime el texto y se ejecutan los comandos que haya desde **THEN** hasta **ELSE**. En caso contrario, se imprime el texto y se ejecutan los comandos que haya desde **ELSE** hasta **ENDIF**

- **IF condexpression THEN ... ELSEIF condexpression THEN [... ELSEIF condexpression THEN] ... ELSE ... ENDIF**

Si la expresión condicional *condexpression1* resulta cierta, se imprime el texto y se ejecutan los comandos que haya desde **THEN** hasta **ELSE**. En caso contrario, se verifica si la expresión condicional *condexpression2* se cumple, en cuyo caso imprime el texto y se ejecutan los comandos que haya desde **ELSEIF** hasta el **ELSE** u otro **ELSEIF**. Se pueden encadenar varios **ELSEIF** hasta que se llegue a una última cláusula **ELSE**, que se ejecuta si ninguna de las condiciones anteriores se ha cumplido.

- **WHILE (condexpression) ... WEND**

Se repiten repetidamente el texto y los comandos que haya hasta el **WEND** mientras la condición *condexpression* evalúe a cierto. La evaluación se realiza al principio de cada iteración.

- **DO ... UNTIL (condexpression)**

Se repiten repetidamente el texto y los comandos que haya desde **DO** hasta el **UNTIL** mientras la condición *condexpression* evalúe a falso. La evaluación se realiza al final de cada iteración.

- **FOR variable = inicio TO fin [STEP paso] ... NEXT [variable]**

Bucle contado: repite el texto y los comandos hasta **NEXT** recorriendo *variable* desde *inicio* hasta *fin*. El **STEP** (paso) es **opcional** y por defecto vale 1; puede ser negativo (**STEP -2**) para contar hacia atrás. El paso debe ser una **constante** (no una variable ni una constante **CONST**), porque el compilador necesita conocer la dirección del bucle. La variable del contador va **pelada** (como en **SET/LET**); dentro del cuerpo se lee su valor con **@variable**. El **NEXT** puede llevar opcionalmente el nombre de la variable (**NEXT i**), que debe coincidir. Los límites *inicio* y *fin* pueden ser expresiones (incluso variables); *fin* se re-evalúa en cada iteración. Si al empezar el contador ya ha pasado el límite, el bucle se ejecuta **cero veces** (semántica **BASIC**). Ejemplo:

```
[[
DECLARE 0 AS i
FOR i = 1 TO 5          /* i = 1, 2, 3, 4, 5 */
  PRINT @i
NEXT
FOR i = 10 TO 0 STEP -2 /* cuenta atrás: 10, 8, 6, 4, 2, 0 */
```

```
PRINT @i
NEXT
]]
```

El contador es un **byte** (0..255). El bucle está protegido internamente contra el desbordamiento del byte, de modo que `FOR i = 0 TO 255` y `FOR i = 10 TO 0 STEP -1` funcionan correctamente (no entran en bucle infinito). Ten en cuenta, eso sí, que los límites deben caber en un byte.

Como se puede ver, muchos de estas funciones se ejecutan dependiendo de si se cumple una condición. Por ejemplo:

```
[[IF @var = 0 THEN GOTO salto ENDIF]]
```

Para que se ejecute el salto, tiene que cumplirse que la variable *var* sea igual a cero. Esto es lo que se llama una expresión condicional.

Una condición es siempre una comparación entre dos expresiones numéricas:

- Igual que: `IF @variable = 0 THEN GOTO salto ENDIF`
- Mayor que: `IF @variable > 0 THEN GOTO salto ENDIF`
- Menor que: `IF @variable2 < @variable THEN GOTO salto ENDIF`
- Distinto que: `IF @variable <> 0 THEN GOTO salto ENDIF`
- Mayor o igual que: `IF @variable <= 0 THEN GOTO salto ENDIF`
- Menor o igual que: `IF @variable >= 0 THEN GOTO salto ENDIF`

Además, las condiciones se pueden combinar formando expresiones condicionales mediante operaciones lógicas. Por ejemplo:

```
[[IF (@variable = 0 AND @variable2 = 1) AND NOT @variable3 = 1 THEN GOTO salto ENDIF]]
```

- **Cond1 AND Cond2**: Cierto si se cumple tanto como Cond1 como Cond2.
- **Cond1 OR Cond2**: Cierto si se cumple Cond1 o Cond2.
- **NOT Cond1**: Cierto si la condición Cond1 es falsa.

Recuerdo de nuevo que los operadores lógicos de las expresiones condicionales **no son los mismos que los operadores binarios**.

Además, para aquellos que tengan nociones avanzadas de programación, las condiciones no son “cortocircuitantes”, es decir, se evalúan todas las operaciones lógicas y comparaciones hasta el final.

1.9 Asignaciones e indirección

Las asignaciones y expresiones numéricas ya las hemos visto en [su sección](#), pero vamos a explicar más detalladamente su funcionamiento con la indirección, lo cual merece una sección aparte.

La indirección se realiza poniendo entre corchetes simples una expresión numérica, por ejemplo `[@var+1]`. Lo que estamos queriendo decir es que **nos vamos a referir al variable o variable con el número calculado por la expresión numérica encerrada entre corchetes**. Vamos a verlo con ejemplos:

- `SET var to [2+2]`: indica que vamos a asignar a *var* el contenido del variable `2+2`, es decir 4. Esto es equivalente a `SET var T0 @4`, pero nos permite ir afianzando el concepto.
- `SET var T0 [@indice]`: aquí es donde se muestra realmente la utilidad, ya que estamos indicando que guardamos en *var* el contenido de la variable cuyo número corresponde con el contenido de la variable *indice*. Es decir, podemos cambiar de forma dinámica la variable a la cual hacemos la asignación.
- `SET var T0 [@indice+2]`: la indirección soporta cualquier expresión numérica, esto significa que guardamos en *var* el contenido de la variable corresponda con el contenido de *indice* mas dos.
- `SET [var] T0 0`: La indirección también puede usarse en el lado derecho, y en este caso, no se soportan expresiones numéricas, sólo el número o el identificador de la variable si se ha declarado. Esto significa “asigna cero a la variable cuyo índice corresponda con el contenido de la variable *var*”.
- `INK [@var]`: La indirección también es soportada por aquellos comandos que admitan expresiones numéricas.
- `SET [@@var] T0 0`: Por completitud, ya que operamos con punteros, usando `@@` tenemos la operación complementaria a la indirección, es decir, devuelve el número de índice de una variable dada, con lo que en el ejemplo, la operación sería equivalente a `SET var T0 0`.

Si usamos los indicadores numéricos con las variables (p.ej. `@0`) no resulta muy útil. Su verdadera utilidad reside en usarlo con los identificadores de variables, tal que si tenemos:

```
[[
DECLARE 20 AS var : DECLARE 10 AS index: SET index AS @@var : SET [index] AS 0
]]
```

Vemos que esto nos permite inicializar variables que vayamos a usar para indirección con el identificador de otra variable.

La indirección constituye una herramienta poderosa que nos permite implementar arrays en los variables, pero ¡jojo, que sólo tenemos 256! Además, no hay que confundirlos con los arrays descritos en la siguiente sección, que son de una naturaleza diferente.

Por último, podemos realizar asignaciones múltiples mediante la siguiente construcción:

```
[[
DECLARE 10 AS var
SET var AS {0, 1, 2, 3}
]]
```

Esto sería equivalente a:

```
[[
SET 10 AS 0
SET 11 AS 1
SET 12 AS 2
SET 13 AS 3
]]
```

Es decir, podemos asignar de forma consecutiva una secuencia de valores a una secuencia de variables,

comenzando por la variable indicada en el lado izquierdo. Lo cual permite asignar múltiples valores de forma consecutiva.

1.10 Constantes

A lo largo del desarrollo, puede que necesitemos tener valores con una denominación significativa. De la misma forma que con las variables, podemos darles nombre con **CONST nombre = valor**, y luego usar **nombre** en su lugar. Por ejemplo:

```
[[
  CONST maxBalas = 10      /* declaramos la constante maxBalas con valor 10 */
  PRINT maxBalas           /* imprimimos el valor 10 */
]]
```

Al contrario que las variables, podemos usar tantas constantes como queramos, ya que durante el proceso de compilación se sustituirán por su valor correspondiente.

El nombre de la constante no puede coincidir con el de otra variable, etiqueta o alguno de los comandos.

1.11 Arrays o “secuencias”

En la sección anterior hemos descrito el uso de la indirección para usar las variables como un array, pero CYD también dispone de una forma alternativa de crear arrays de datos mediante el comando **DIM**, de tal manera que con **DIM nombre(tamaño)** declaramos un array de nombre **nombre** y tamaño **tamaño**. El tamaño de un array no puede ser mayor de 256 ni ser cero. Accedemos a cada uno de los elementos del array con la nomenclatura **nombre(pos)**, donde **pos** es el número de posición del elemento a acceder dentro del array, empezando desde cero. Vamos a verlo con ejemplos:

```
[[
  DIM miArray(5)           /* Declaramos un array de 5 elementos del 0 al 4 */
  LET miArray(3) = 1        /* Asignamos 1 a la posición 3 del array, que sería la cuarta */
  PRINT miArray(3)          /* Imprimimos el valor almacenado en la posición 3 */
]]
```

Si intentamos acceder a una posición fuera del rango, el motor fallará y dará un error. Para saber cual es la última posición accesible del array, tenemos la función **LASTPOS**:

```
[[
  DIM miArray(5)           /* Declaramos un array de 5 elementos */
  PRINT LASTPOS(miArray)    /* Devolvería 4 */
]]
```

Asignar valores iniciales a un array se puede hacer de la siguiente forma: **DIM nombre_array(tamaño) = {valor1, valor2, ...}**. Siguiendo el ejemplo anterior:

```
[[
  DIM miArray(5) = {1, 2, 3, 4, 5} /* Asignamos valores al array */
  PRINT miArray(1)                  /* Esto imprimiría 2 */
]]
```

```
]]
```

Las posiciones no inicializadas siempre tendrán valor 0 por defecto, con lo que podemos tener un array con más posiciones que valores. Lo contrario nos daría un error:

```
[[
  DIM miArray(5) = {1, 2, 3}      /* Esto se permite */
  DIM miArray(3) = {1, 2, 3, 4, 5} /* Esto daría error */
]]
```

Por último, y como conveniencia, podemos dejar vacío el número de posiciones del array al declararlo con valores. El compilador creará un array de tantas posiciones como elementos haya en la inicialización:

```
[[
  DIM miArray() = {1, 2, 3} /* Esto se permite, miArray sería de 3 posiciones */
  DIM miArray()           /* Esto daría error */
]]
```

Este tipo arrays tienen una serie de limitaciones que hay que tener en cuenta a la hora de usarlos. La más evidente es que no pueden tener más de 256 elementos y son unidimensionales y no se pueden redimensionar.

Otra limitación es que no se pueden salvar ni recuperar los datos de estos arrays mediante **SAVE** y **LOAD** directamente. La única opción es mover los datos del array desde o hacia las variables y realizar las operaciones de grabar o recuperar en las mismas.

Por último, si se modifican los datos de un array, no se pueden recuperar los valores originales a menos que se haga una copia en otro array previamente.

1.12 Datos inmutables (**DATA / READ / RESTORE / DATAEND()**)

Cuando lo que necesitamos es un bloque de datos de **solo lectura** (tablas, mapas, textos comprimidos byte a byte, secuencias...), CYD ofrece el mecanismo **DATA**, análogo al de BASIC. A diferencia de los arrays **DIM** —que son escribibles y están limitados a 256 elementos—, los **DATA** son **inmutables** y forman un **único flujo global** que puede llegar hasta **16 KB** (más de 16000 elementos). Al no modificarse nunca, ocupan **0 bytes de RAM en Dandanator/MLD** (se leen directamente de la flash).

Cada sentencia **DATA** añade sus bytes, en el orden en que aparecen en el fuente, a ese flujo global. Se leen secuencialmente con **READ**, que usa el mismo destino “pelado” que **SET/LET**:

```
[[
  DECLARE 0 AS v

  DATA 10, 20, 30      /* estos cinco valores forman un único flujo: */
  DATA 40, 50          /* 10, 20, 30, 40, 50 */

  READ v                /* v = 10, y el cursor avanza */
  READ [v]              /* destino indirecto (índice en v), como LET [v] */
]]
```

Hay un **único cursor global** que empieza al principio del flujo. `RESTORE` lo rebobina al principio, y `RESTORE` etiqueta lo coloca en el **primer DATA que aparezca tras esa etiqueta** (misma semántica que el `RESTORE` línea de `BASIC`; se reutilizan las etiquetas `LABEL`):

```
[[
  DECLARE 0 AS v

  DATA 11, 22, 33
  LABEL segundo
  DATA 44, 55

  RESTORE segundo      /* el cursor se coloca en el '44' */
  READ v               /* v = 44 */
]]
```

La función `DATAEND()` devuelve 1 si el cursor ha llegado al final del flujo y 0 en caso contrario. Es útil para recorrer todos los datos con un bucle (nótese que es una función, con paréntesis, y que para usarla como condición se compara, p. ej. `DATAEND() = 0`):

```
[[
  DECLARE 0 AS v

  DATA 65, 66, 67

  WHILE (DATAEND() = 0) /* mientras no lleguemos al final */
    READ v
    CHAR v              /* imprime el carácter A, B, C */
  WEND
]]
```

Consideraciones a tener en cuenta:

- Los **DATA no se ejecutan**: pueden colocarse en cualquier punto del guion (el compilador los recoge todos y los saca del flujo de código). Por claridad conviene agruparlos.
- **Cinta sin fin**: leer más allá del final rebobina automáticamente el cursor al principio del flujo (no da error). Usa `DATAEND()` si quieres detectar el final y parar.
- Los valores son constantes de byte (aceptan las mismas expresiones y constantes con nombre que la inicialización de un `DIM`).
- Si un programa no usa `DATA`, los opcodes correspondientes se eliminan del intérprete (coste 0).

1.13 Constantes anchas (`WORD` / `DWORD` / cadenas)

CYD trabaja con bytes, pero a menudo necesitas colocar en memoria valores de **16 bits** (0..65535) o **32 bits** (compatibles con la librería `math16_32`), o el código de los caracteres de una **cadena**. Para eso están las constantes anchas: valores que el compilador expande a una secuencia **fija de bytes consecutivos en formato *little-endian*** (byte bajo primero). Todo ocurre en tiempo de compilación: **no añaden ningún runtime**.

Se pueden usar en tres sitios: en **DATA**, en la inicialización de un **DIM** y en **LET/SET** de una variable.

- **Autodetección por magnitud** (solo en **DATA** y **DIM**): un valor de 0 a 255 ocupa 1 byte, de 256 a 65535 ocupa 2 bytes y de 65536 en adelante ocupa 4 bytes, automáticamente.
- **Marcadores WORD y DWORD**: fuerzan 2 o 4 bytes aunque el valor quepa en menos (**WORD** 5 = 2 bytes). **WORD** no admite valores mayores de 16 bits y **DWORD** no admite mayores de 32 bits.
- **Cadenas "..."**: se expanden a los códigos de sus caracteres (sin terminador; tú conoces la longitud porque la escribiste).

```
[[
  DATA WORD 1000, 5, "HI"           /* 2 bytes + 1 byte + 2 bytes */
  DIM tabla() = { DWORD 100000, 42 } /* 4 bytes + 1 byte -> array de 5 bytes */
]]
```

Los arrays quedan **byte a byte** (*byte-plano*): no hay acceso “ancho” a un array, eres tú quien recompone el valor leyendo los bytes que necesites (con `math16_32` si vas a operar).

En **LET/SET**, el ancho lo marca **siempre el keyword** (**WORD/DWORD**) o la cadena; los bytes se escriben en la variable de destino y las siguientes consecutivas. Aquí **no hay autodetección** (el número de variables ocupadas debe conocerse al compilar, así que un `LET v = 1000` sin marcador seguiría siendo de 1 byte y daría error por no caber):

```
[[
  DECLARE 0 AS puntos    /* ocupará las variables 0 y 1 */
  DECLARE 2 AS nombre    /* ocupará las variables 2 y 3 */
  LET puntos = WORD 1000 /* var 0 = byte bajo (232), var 1 = byte alto (3) */
  LET nombre = "AB"      /* var 2 = cód. 'A', var 3 = cód. 'B' */
]]
```

Solo se admite destino **directo** (índices consecutivos conocidos): la indirección `LET [v]` no se puede usar con constantes anchas.

1.14 Literales cómodos (carácter, rangos, repetición)

Para no memorizar códigos ni escribir secuencias largas a mano, CYD ofrece varios literales que el compilador expande en tiempo de compilación (0 runtime):

- **Literal de carácter 'A'** → el **código de glifo** del carácter. Vale en cualquier sitio donde vaya un número (**DATA**, **DIM**, **LET**, expresiones, **CASE**...). Ej.: `LET tecla = 'A'` equivale a `LET tecla = 65`.
- **Rangos {a..b}** → la secuencia de bytes *a*, *a+1*, ..., *b* (o descendente si *a > b*). Los extremos son constantes de byte y admiten caracteres: `'A'..'Z'`. Ej.: `DIM cuenta() = { 1..8 }`.
- **Repetición { v REPEAT n }** → *n* copias de *v* (que puede ser un valor, una constante ancha, una cadena o un rango). Ej.: `DIM buffer() = { 0 REPEAT 16 }` (dieciséis ceros).

Rangos y repetición solo tienen sentido en contextos de **lista** (**DATA**, inicialización de **DIM**), y se combinan entre sí: `DATA 255 REPEAT 4, 1..3, 'A'`.

1.15 Listado de comandos

Antes de describir los comandos, vamos a hablar resumidamente de la leyenda utilizada:

- **ID** es un identificador y es una cadena de cifras, letras o subrayado. No se admiten letras acentuadas ni la ñ.
 - **expression** es una expresión numérica sin variables. Es un número en decimal o hexadecimal, pero también se admiten operaciones aritméticas simples que serán calculadas en tiempo de compilación.
 - **varexpression**, expresión numérica tal y como es descrita en su [sección](#) correspondiente. Puede contener variables con @ e indirectones.
 - **condexpression**, expresión condicional tal y como es descrita en su [sección](#) correspondiente. Consisten en una comparación o varias comparaciones con operaciones lógicas entre las mismas.
 - **varID**, identificador de variable, puede ser una *expression* si usamos su índice numérico o un *ID* si lo hemos declarado así con **DECLARE**.
-

Este es listado completo de comandos:

1.15.1 INCLUDE “ruta/al/archivo.cyd”

Directiva de compilación que inserta otro archivo *.cyd* en el punto actual. Debe usarse dentro de bloques `[ [ ] ]`. Se resuelve antes del parseo, admite inclusiones anidadas y previene referencias circulares.

1.15.2 LABEL ID

Declara la etiqueta *labelId* en este punto. Todos los saltos con referencia a esta etiqueta dirigirán la ejecución a este punto.

1.15.3 #ID

Versión resumida para declarar la etiqueta *ID*, de tal manera que *#LabelId* es lo mismo que **LABEL** *LabelId*.

1.15.4 DECLARE expression AS ID

Declara el identificador *ID* como un símbolo que representa al variable *expression* en su lugar.

1.15.5 CONST ID = expression

Declara la constante *ID* con el valor de *expression*.

1.15.6 DIM ID(expression)

Crea un array de nombre *ID* con tantos elementos como se indique en *expression*. El número de elementos no puede ser mayor de 256.

1.15.7 DIM ID(expression) = {expression, expression...}

Igual que **DIM ID(expression)**, pero asignando valores separados por comas. Los elementos pueden ser constantes de byte, **constantes anchas** (**WORD/DWORD** o autodetectadas) o cadenas; el array queda byte a byte (ver Constantes anchas).

1.15.8 DATA expression, expression...

Añade los valores al flujo global de datos inmutables de solo lectura, en el orden en que aparecen en el fuente. Todas las sentencias **DATA** del guion forman un único flujo (máximo 16 KB). No se ejecutan; el compilador las extrae del código. Cada elemento puede ser una constante de byte, una **constante ancha** (WORD/DWORD o autodetectada por magnitud) o una cadena; ver Constantes anchas.

1.15.9 READ varID

Lee el siguiente byte del flujo de datos a la variable *varID* y avanza el cursor.

1.15.10 READ [varID]

Igual que [READ varID](#), pero el destino es la variable cuyo índice corresponde con el contenido de *varID* (destino indirecto).

1.15.11 RESTORE

Rebobina el cursor del flujo de datos al principio.

1.15.12 RESTORE ID

Coloca el cursor del flujo de datos en el primer **DATA** que aparezca tras la etiqueta *ID* en el fuente.

1.15.13 DATAEND()

Devuelve 1 si el cursor del flujo de datos ha llegado al final, 0 en caso contrario.

1.15.14 GOTO ID

Salta a la etiqueta *ID*.

1.15.15 GOSUB ID

Salto de subrutina, hace un salto a la etiqueta *ID*, pero vuelve al siguiente comando cuando encuentra un comando **RETURN**.

1.15.16 RETURN

Retorna al punto posterior de la llamada de la subrutina, ver **GOSUB**

1.15.17 IF condexpression THEN ... ENDIF

Si la expresión condicional *condexpression* resulta cierta, se imprime el texto y se ejecutan los comandos que haya desde **THEN** hasta **ENDIF**.

1.15.18 IF condexpression THEN ... ELSE ... ENDIF

Si la expresión condicional *condexpression* resulta cierta, se imprime el texto y se ejecutan los comandos que haya desde **THEN** hasta **ELSE**. En caso contrario, se imprime el texto y se ejecutan los comandos que haya desde **ELSE** hasta **ENDIF**

1.15.19 IF condexpression1 THEN ... ELSEIF condexpression2 THEN ... ELSE ... ENDIF

Si la expresión condicional *condexpression1* resulta cierta, se imprime el texto y se ejecutan los comandos que haya desde **THEN** hasta **ELSE**. En caso contrario, se verifica si la expresión condicional *condexpression2* se cumple, en cuyo caso imprime el texto y se ejecutan los comandos que haya desde **ELSEIF** hasta el **ELSE** u otro **ELSEIF**. Se pueden encadenar varios **ELSEIF** hasta que se llegue a

una última cláusula **ELSE**, que se ejecuta si ninguna de las condiciones anteriores se ha cumplido.

1.15.20 WHILE (condexpression) ... WEND

Se repiten repetidamente el texto y los comandos que haya hasta el **WEND** mientras la condición *condexpression* evalúe a cierto. La evaluación se realiza al principio de cada iteración.

1.15.21 DO ... UNTIL (condexpression)

Se repiten repetidamente el texto y los comandos que haya desde **DO** hasta el **UNTIL** mientras la condición *condexpression* evalúe a falso. La evaluación se realiza al final de cada iteración.

1.15.22 SELECT varexpression ... CASE valor[,valor...] ... [CASE ELSE ...] ENDSELECT

Selección múltiple: evalúa *varexpression* y ejecuta la rama del **CASE** cuyo valor coincida (**CASE** 1, 2, 3 = varios valores para la misma rama). **CASE ELSE** (opcional) es la rama por defecto. **Sin fall-through**: al acabar una rama, salta al final (semántica BASIC). El sujeto se re-evalúa en cada comparación (como el límite del **FOR**), así que conviene que sea una variable o una expresión sin efectos secundarios. Ver [control de flujo](#).

1.15.23 ENUM [nombre] { A, B=valor, C, ... }

Declara constantes secuenciales (como varios **CONST**): auto-incremento desde 0, o desde el último valor explícito (estilo C). **ENUM** { NORTE, SUR, ESTE } → 0, 1, 2. Los nombres son **constantes globales**; el *nombre* opcional es solo agrupación/legibilidad. Los valores explícitos deben ser literales numéricos.

1.15.24 SWAP varID, varID

Intercambia el contenido de dos variables sin usar una temporal. Operandos **pelados** (son destinos). También admite destino **indirecto** en cada lado: **SWAP** [a], [b], **SWAP** a, [b].

1.15.25 FOR varID = varexpression TO varexpression [STEP expression] ... NEXT [varID]

Bucle contado: recorre *varID* desde el valor inicial hasta el final. **STEP** es opcional (por defecto 1) y debe ser una **constante** (puede ser negativa). El contador es un byte y va pelado; en el cuerpo se lee con @*varID*. Si el inicio ya ha pasado el límite, se ejecuta cero veces. Ver la sección de [control de flujo](#) para más detalles.

1.15.26 SET varID TO varexpression

Asigna el valor de *varexpression* a la variable *varID*.

1.15.27 SET varID TO WORD expression / DWORD expression / “cadena”

Asigna una **constante ancha** a *varID* y las variables consecutivas: **WORD** escribe 2 bytes (little-endian), **DWORD** 4 bytes y una cadena sus códigos de carácter. Ver Constantes anchas. (También válido con **LET** *varID* =)

1.15.28 SET [varID] TO varexpression

Asigna el valor de *varexpression* a la variable cuyo índice corresponde con el contenido de *varID*.

1.15.29 SET varID TO {varexpression1, varexpression2,...}

Asigna el valor de *varexpression1* a la variable *varID*, *varexpression2* a la variable *varID*+1, y así.

1.15.30 SET [varID] TO {varexpression1, varexpression2,...}

Asigna el valor de *varexpression1* a la variable cuyo índice corresponde con el contenido de *varID*, *varexpression2* a la variable cuyo índice corresponde con el contenido de *varID*+1, y así.

1.15.31 LET varID = varexpression

Asigna el valor de *varexpression* a la variable *varID*.

1.15.32 LET [varID] = varexpression

Asigna el valor de *varexpression* a la variable cuyo índice corresponde con el contenido de *varID*.

1.15.33 LET varID = {varexpression1, varexpression2,...}

Asigna el valor de *varexpression1* a la variable *varID*, *varexpression2* a la variable *varID*+1, y así.

1.15.34 LET [varID] = {varexpression1, varexpression2,...}

Asigna el valor de *varexpression1* a la variable cuyo índice corresponde con el contenido de *varID*, *varexpression2* a la variable cuyo índice corresponde con el contenido de *varID*+1, y así.

1.15.35 LET varID += varexpression

Incrementa la variable *varID* en el valor de *varexpression*. Equivalente a LET varID = @varID + varexpression.

1.15.36 LET [varID] += varexpression

Incrementa la variable cuyo índice corresponde con el contenido de *varID* en el valor de *varexpression*.

1.15.37 LET arrayID(varexpression) += varexpression

Incrementa el elemento de la posición *varexpression* del array *arrayID* en el valor de la segunda *varexpression*.

1.15.38 LET varID -= varexpression

Decrementa la variable *varID* en el valor de *varexpression*. Equivalente a LET varID = @varID - varexpression.

1.15.39 LET [varID] -= varexpression

Decrementa la variable cuyo índice corresponde con el contenido de *varID* en el valor de *varexpression*.

1.15.40 LET arrayID(varexpression) -= varexpression

Decrementa el elemento de la posición *varexpression* del array *arrayID* en el valor de la segunda *varexpression*.

1.15.41 END

Finaliza la aventura y reinicia el ordenador.

1.15.42 CLEAR

Borra el área de texto definida.

1.15.43 CENTER

Pone el cursor de impresión en el centro de la línea.

1.15.44 WAITKEY

Espera la pulsación de la tecla de aceptación para continuar, presentando un icono animado de espera. Ideal para separar párrafos o pantallas.

1.15.45 OPTION GOTO ID

Crea un punto de opción que el usuario puede seleccionar (ver **CHOOSE**). Si confirma esta opción, salta a la etiqueta *ID*. Si se borra la pantalla, el punto de opción se elimina y sólo se permiten 32 como máximo en una pantalla. Los puntos de opción se van acumulando en orden de declaración en una lista que será recorrida de acuerdo a la pulsación de las teclas de manejo, y en el mismo orden en el que se declararon los puntos de opción. Si resulta la opción seleccionada en el menú, el valor devuelto por **OPTIONVAL()** es su posición dentro de la lista de opciones del menú.

1.15.46 OPTION GOSUB ID

Crea un punto de opción que el usuario puede seleccionar (ver **CHOOSE**). Si confirma esta opción, hace un salto de subrutina a etiqueta *ID*, volviendo después del **CHOOSE** cuando encuentra un **RETURN**. Si se borra la pantalla, el punto de opción se elimina y sólo se permiten 32 como máximo en una pantalla. Los puntos de opción se van acumulando en orden de declaración en una lista que será recorrida de acuerdo a la pulsación de las teclas de manejo, y en el mismo orden en el que se declararon los puntos de opción. Si resulta la opción seleccionada en el menú, el valor devuelto por **OPTIONVAL()** es su posición dentro de la lista de opciones del menú.

1.15.47 OPTION VALUE(varexpression) GOTO ID

Funciona igual que **OPTION GOTO ID**, pero le asignamos un valor específico que será lo que devuelva **OPTIONVAL()** si resulta la opción elegida en el menú.

1.15.48 OPTION VALUE(varexpression) GOSUB ID

Funciona igual que **OPTION GOSUB ID**, pero le asignamos un valor específico que será lo que devuelva **OPTIONVAL()** si resulta la opción elegida en el menú.

1.15.49 CHOOSE

Detiene la ejecución y permite al jugador seleccionar una de las opciones que haya en este momento en pantalla. Realizará el salto a la etiqueta indicada en la opción correspondiente. La selección se realiza con las teclas **O** y **P** para “desplazamiento horizontal” y **Q** y **A** para desplazamiento vertical. También se pueden usar las direcciones correspondientes del joystick Kempston y las teclas del cursor. Cuando se pulsan las teclas, un puntero se desplaza sobre la lista de opciones realizando incrementos o decrementos de acuerdo a la configuración actual del mismo (ver **MENUCONFIG**). Por defecto, sólo tiene configurado un desplazamiento vertical con las teclas **Q** y **A** ó **arriba** y **abajo** en el joystick Kempston o teclas de cursor. Es responsabilidad del usuario colocar los puntos de opción en pantalla de una forma coherente al movimiento del menú configurado.

Recuerda que si se borra la pantalla antes de este comando, perderás las opciones y dará un error de sistema.

1.15.50 CHOOSE IF WAIT expression THEN GOTO ID

Funciona exactamente igual que **CHOOSE**, pero con la salvedad de que se declara un timeout, que si se agota sin seleccionar ninguna opción, salta a la etiqueta *ID*. El timeout tiene como máximo 65535 (16 bits).

1.15.51 CHOOSE IF CHANGED THEN GOSUB ID

Funciona exactamente igual que CHOOSE, pero se hace un salto de subrutina a la etiqueta *ID* cada vez que se cambia de opción. Por tanto, es recomendable que se haga un RETURN lo antes posible para evitar lentitud. Para saber la opción elegida y el número de opciones total de nuestro menú, podemos usar las funciones OPTIONSEL() y NUMOPTIONS respectivamente.

1.15.52 OPTIONSEL()

Función que devuelve la opción actualmente seleccionada en el menú actualmente activo. Si no tenemos un menú activo, el resultado de esta función está indefinido.

1.15.53 NUMOPTIONS()

Función que devuelve el número de opciones del menú actualmente activo. Si no tenemos un menú activo, el resultado de esta función está indefinido.

1.15.54 OPTIONVAL()

Función que devuelve el valor asignado a la opción seleccionada y aceptada en el menú, sólo se actualiza cuando una opción es elegida y se sale del menú.

1.15.55 CLEAROPTIONS

Elimina las opciones almacenadas en el menú.

1.15.56 MENUCONFIG varexpression, varexpression, varexpression, varexpression

Configura el menú de opciones. Los parámetros que se pueden configurar son los siguientes:

- El primer parámetro determina el incremento o decremento del número de opción seleccionado cuando pulsamos **P** y **O** respectivamente (o las teclas de izquierda y derecha del cursor o del joystick).
- El segundo parámetro determina el incremento o decremento del número de opción seleccionado cuando pulsamos **A** y **Q** respectivamente (o las teclas de arriba y abajo del cursor o del joystick).
- El tercer parámetro es la opción que se seleccionará al principio cuando se inicie el menú con CHOOSE. El valor por defecto es cero (la primera opción registrada).
- El cuarto parámetro, si es cero no muestra el icono de selección y si es distinto de cero, lo muestra.

El comportamiento al iniciarse el intérprete es como si se hubiese ejecutado MENUCONFIG 0,1,0,1. Si se cambia la configuración para un menú, es recomendable hacerlo lo primero, antes de situar las opciones.

1.15.57 MENUCONFIG varexpression, varexpression, varexpression

Si se omite el cuarto parámetro, MENUCONFIG x,y,d equivale a MENUCONFIG x,y,d,1.

1.15.58 MENUCONFIG varexpression, varexpression

Si se omiten el tercer parámetro y cuarto parámetros, MENUCONFIG x,y equivale a MENUCONFIG x,y,0,1.

1.15.59 CHAR varexpression

Imprime el carácter indicando con su número correspondiente.

1.15.60 REPCCHAR expression, expression

Imprime el carácter indicado en el primer parámetro tantas veces como número se indique en el segundo parámetro. Ambos valores tienen un tamaño de 1 byte, es decir, val del 0 al 255. Además, si el número de veces es cero, el carácter se repetirá 256 veces en lugar de ninguna.

1.15.61 TAB expression

Imprime tantos espacios como los indicados en el parámetro. Es el equivalente a `REPCCHAR 32, expression`, pero ocupa menos memoria.

1.15.62 PRINT varexpression

Imprime el valor numérico indicado.

1.15.63 PAGEPAUSE expression

Controla si al rellenar por completo el área de texto actual, debe solicitar continuar al jugador, presentando un icono animado de espera (parámetro `<> 0`) ó hace un desplazamiento hacia arriba del texto y sigue imprimiendo (parámetro `= 0`).

1.15.64 INK varexpression

Define el valor del color de los caracteres (tinta). Valores de 0-7, correspondientes a los colores del Spectrum.

1.15.65 PAPER varexpression

Define el valor del color del fondo (papel). Valores de 0-7, correspondientes a los colores del Spectrum.

1.15.66 BORDER varexpression

Define el color del borde, valores 0-7.

1.15.67 BRIGHT varexpression

Activa o desactiva el brillo (0 desactivado, 1 activado).

1.15.68 FLASH varexpression

Activa o desactiva el parpadeo (0 desactivado, 1 activado).

1.15.69 NEWLINE expression

Imprime tantos saltos de línea como el número indicado en el parámetro (0 es 256)

1.15.70 NEWLINE

Equivalente a `NEWLINE 1`.

1.15.71 BACKSPACE expression

Retrasa el cursor de impresión tantas posiciones como las indicadas en el parámetro (0 es 256), eliminado el carácter en la nueva posición y sustituyéndolo por un espacio. El tamaño de carácter que se usará es el del espacio (número 32). Hay que tener en cuenta que si se usan caracteres de distintos tamaños al del espacio, esta operación no funcionará bien.

1.15.72 BACKSPACE

Equivalente a `BACKSPACE 1`.

1.15.73 SFX varexpression

Si se ha cargado un fichero de efectos de sonido, reproduce el efecto indicado. Si no se ha cargado dicho fichero, el comando es ignorado.

1.15.74 PICTURE varexpression

Carga en el buffer la imagen indicada como parámetro. Por ejemplo, si se indica 3, cargará el fichero 003.CSC. La imagen no se muestra, lo que permite controlar cuándo se realiza la carga del fichero.

1.15.75 DISPLAY varexpression

Muestra el contenido actual del buffer en pantalla. El parámetro indica si se muestra o no la imagen, con un 0 no se muestra, y con un valor distinto de cero, sí. Se muestran tantas líneas como se hayan definido en la imagen correspondiente y el contenido de la pantalla será sobrescrito.

1.15.76 BLIT varexpression, varexpression, varexpression, varexpression AT varexpression, varexpression

Copia una parte de la imagen cargada en el buffer a la pantalla. Definimos con los parámetros un rectángulo dentro del buffer que se copiará en la pantalla a partir de la posición indicada.

Los parámetros, por orden, son:

- Columna origen del rectángulo a copiar desde el buffer.
- Fila origen del rectángulo a copiar desde el buffer.
- Ancho del rectángulo a copiar desde el buffer.
- Alto del rectángulo a copiar desde el buffer.
- Columna destino en pantalla.
- Fila destino en pantalla.

La unidad de medida empleada en todos los parámetros es el carácter 8x8 y sólo los dos últimos aceptan variables.

1.15.77 WAIT expression

Realiza una pausa. El parámetro es el número de “fotogramas” (50 por segundo) a esperar, y es un número de 16 bits.

1.15.78 PAUSE expression

Igual que WAIT, pero con la salvedad de que el jugador puede abortar la pausa con la pulsación de la tecla de confirmación.

1.15.79 TYPERATE expression

Indica la pausa que debe haber entre la impresión de cada carácter. Mínimo 0, máximo 65535.

1.15.80 MARGINS expression, expression, expression, expression

Define el área de pantalla donde se escribirá el texto. Los parámetros, por orden, son:

- Columna inicial.
- Fila inicial.
- Ancho (en caracteres).
- Alto (en caracteres).

Los tamaños y posiciones siempre se definen como si fuesen caracteres 8x8.

1.15.81 FADEOUT *expression, expression, expression, expression*

Hace un fundido en negro en el área de pantalla definida por los parámetros que, por orden, son:

- Columna inicial.
- Fila inicial.
- Ancho (en caracteres).
- Alto (en caracteres).

Los tamaños y posiciones siempre se definen como si fuesen caracteres 8x8.

1.15.82 AT *varexpression, varexpression*

Sitúa el cursor en una posición dada, relativa al área definida por el comando MARGINS. Los parámetros, por orden, son:

- Columna relativa al origen del área de texto.
- Fila relativa al origen del área de texto.

Las posiciones se asumen en tamaño de carácter 8x8.

1.15.83 FILLATTR *varexpression, varexpression, varexpression, varexpression, varexpression*

Rellenamos en pantalla un rectángulo con un valor de atributos determinado. Los píxeles no se alteran. Los parámetros, por orden, son:

- Columna origen del rectángulo a rellenar.
- Fila origen del rectángulo a rellenar.
- Ancho del rectángulo a rellenar.
- Alto del rectángulo a rellenar.
- Valor de atributos en formato de pantalla de Spectrum, es decir, un byte con bits en formato FBPPPIII (F = Flash, B = Brillo, P = Papel, I = Tinta).

1.15.84 PUTATTR *varexpression, varexpression AT varexpression, varexpression*

Ponemos los atributos de un carácter 8x8 en pantalla con unos valores determinados. Los píxeles no se alteran. Los parámetros, por orden, son:

- Valor de atributos en formato de pantalla de Spectrum, es decir, un byte con bits en formato FBPPPIII (F = Flash, B = Brillo, P = Papel, I = Tinta).
- Máscara a aplicar sobre los atributos ya existentes en pantalla. Si el bit es cero, conservamos el de la pantalla, y si es uno usamos el nuevo valor.
- Columna del carácter 8x8.
- Fila del carácter 8x8.

1.15.85 PUTATTR *varexpression AT varexpression, varexpression*

Equivalente a PUTATTR *varexpression, 0xFF AT varexpression, varexpression*

1.15.86 GETATTR (varexpression, varexpression)

Función que devuelve el valor de atributo de un carácter 8x8 en pantalla, en formato de pantalla de Spectrum, es decir, un byte con bits en formato FBPPPIII (F = Flash, B = Brillo, P = Papel, I = Tinta).

Los parámetros, por orden, son:

- Columna del carácter 8x8.
- Fila del carácter 8x8.

1.15.87 ATTRVAL (expression COMMA expression COMMA expression COMMA expression)

Función que devuelve el valor de atributos en formato de pantalla de Spectrum, es decir, un byte con bits en formato FBPPPIII (F = Flash, B = Brillo, P = Papel, I = Tinta), utilizado en los comandos PUTATTR y FILLATTR.

Los parámetros, por orden, son:

- Color de tinta (0 a 7).
- Color de fondo (0 a 7).
- Uso de brillo (0 ó 1).
- Uso de parpadeo (0 ó 1).

1.15.88 ATTRMASK (expression COMMA expression COMMA expression COMMA expression)

Función que devuelve un valor de máscara, utilizado en los comandos PUTATTR y FILLATTR.

Los parámetros, por orden, son:

- Máscara de tinta. Si el valor es cero, conservamos el valor de la pantalla, y si es uno usamos el nuevo valor.
- Máscara de fondo. Si el valor es cero, conservamos el valor de la pantalla, y si es uno usamos el nuevo valor.
- Máscara de brillo. Si el valor es cero, conservamos el valor de la pantalla, y si es uno usamos el nuevo valor.
- Máscara de parpadeo. Si el valor es cero, conservamos el valor de la pantalla, y si es uno usamos el nuevo valor.

1.15.89 RANDOM(expression)

Función que devuelve un número aleatorio entre 0 y el valor indicado en **expression**. El máximo es 255.

1.15.90 RANDOM()

Función que devuelve un número aleatorio entre 0 y 255. Es el equivalente a **RANDOM(255)**.

1.15.91 RANDOM(expression, expression)

Función que devuelve un número aleatorio entre el valor indicado en el primer parámetro y el segundo, ambos inclusive.

1.15.92 INKEY(expression)

Función que devuelve el código de la tecla pulsada. Si el parámetro es cero, se espera hasta que se pulse una tecla. Si es distinto de cero, devuelve el código de la tecla pulsada en ese momento, y cero si no hay ninguna tecla pulsada. **Sólo responde a las pulsaciones del teclado, no del joystick**

1.15.93 INKEY()

Función que espera hasta que se pulse una tecla y devuelve el código de la tecla pulsada. Es equivalente a INKEY(0)

1.15.94 KEMPSTON()

Función que devuelve las teclas pulsadas en un joystick kempston conectado. Los bits siguientes estarán a uno si el botón correspondiente está pulsado:

- Bit 0 = Derecha.
- Bit 1 = Izquierda.
- Bit 2 = Abajo.
- Bit 3 = Arriba.
- Bit 4 = Disparo.

1.15.95 MIN(varexpression,varexpression)

Función que acota el valor del primer parámetro al mínimo indicado por el segundo. Es decir, si el parámetro 1 es menor que el parámetro 2, se devuelve el parámetro 2, en otro caso se devuelve el 1.

1.15.96 MAX(varexpression,varexpression)

Función que acota el valor del primer parámetro al máximo indicado por el segundo. Es decir, si el parámetro 1 es mayor que el parámetro 2, se devuelve el parámetro 2, en otro caso se devuelve el 1.

1.15.97 YPOS()

Función que devuelve la fila actual en la que se encuentra el cursor en coordenadas 8x8.

1.15.98 XPOS()

Función que devuelve la columna actual en la que se encuentra el cursor en coordenadas 8x8 (Debido a la naturaleza de la fuente de ancho variable, el valor devuelto será la columna 8x8 donde esté actualmente el cursor).

1.15.99 WINDOW expression

Se cambia a la “ventana” indicada por el parámetro y se soporta hasta 8 ventanas (desde 0 a 7). Por defecto, la ventana inicial es la 0. Las “ventanas” de CYD son un área de pantalla definida por su posición de origen, su ancho, su alto, la posición del cursor y los atributos actuales. Esto permite tener diferentes áreas de texto independientes en pantalla simultáneamente sin tener que estar cambiando márgenes y la posición del cursor continuamente para pasar de un área a otra. Ten en cuenta que si se solapan ventanas el contenido de una ventana no se conserva si se sobrescribe con en otra ventana.

1.15.100 CHARSET expression

Cambia el juego de caracteres empleado al imprimir textos. Con el parámetro a cero, se usa el juego inferior (caracteres del 0 al 127), y con un valor distinto de cero, se usa el juego superior (caracteres

del 128 al 255). Recuerda que los caracteres del 0 al 32 y del 127 al 143 no son imprimibles. Esta instrucción no afecta a REPCHAR y CHAR.

1.15.101 RANDOMIZE expression

Inicializa el generador de números aleatorios con el valor del parámetro como semilla. La generación de números aleatorios no es realmente “aleatoria” y esto puede ocasionar que el generador devuelva siempre los mismos resultados si se usa en un emulador, por lo que se necesita alguna fuente de aleatoriedad o entropía. Si usamos este comando con el valor cero como parámetro, se inicializa el generador usando el número de “frames” o “fotogramas” transcurridos, con lo que si se ejecuta en respuesta a algún evento arbitrario, como la pulsación de una tecla, garantizamos la aleatoriedad. Si usamos un valor distinto de cero, **RANDOM()** siempre dará la misma secuencia de resultados, pero puede servir como generador procedural.

1.15.102 RANDOMIZE

Esto es equivalente a hacer **RANDOMIZE 0**, con lo que el generador de números aleatorios se cargará con el número de frames o fotogramas transcurridos.

1.15.103 TRACK varexpression

Carga en memoria el fichero de Vortex Tracker como parámetro. Por ejemplo, si se indica 3, cargará la pista de música del fichero 003.PT3 (o 003.mus en caso de usar WyzTracker). Si existiese una pista cargada previamente, la sobrescribirá.

1.15.104 PLAY varexpression

Si el parámetro es distinto de cero y la música está desactivada, reproduce la pista musical cargada. Si está en ese momento reproduciendo, y se pasa 0 como parámetro, para la reproducción.

1.15.105 LOOP varexpression

Establece si al acabar la pista musical cargada en ese momento, se repite de nuevo o no. Un valor 0 significa falso y distinto de cero, verdadero. Este comando no funciona con WyzTracker.

1.15.106 RAMSAVE varID, expression

Copia desde la variable indicada en el primer parámetro, tantas variables como las indicadas en el segundo parámetro (si es cero, se considera como 256) a un almacenamiento temporal. Antes de realizar la copia, el almacén temporal se rellena con ceros, con lo que las variables no copiadas tendrán este valor en el almacén temporal.

1.15.107 RAMSAVE varID

Es el equivalente a **RAMSAVE varId, 0**, es decir, copia todas las variables desde el primer parámetro hasta el final al almacén temporal.

1.15.108 RAMSAVE

Es el equivalente a **RAMSAVE 0, 0**, es decir, copia todas las variables al almacén temporal.

1.15.109 RAMLOAD varID, expression

Es la operación inversa a **RAMSAVE**. Copia desde el almacenamiento temporal a las variables, desde la variable indicada en el primer parámetro y tantas variables como las indicadas en el segundo (si es cero, se considera como 256).

1.15.110 RAMLOAD varId

Es el equivalente a `RAMLOAD varId, 0`, es decir, copia todas las variables desde el primer parámetro hasta el final desde el almacén temporal.

1.15.111 RAMLOAD

Es el equivalente a `RAMLOAD 0, 0`, es decir, copia todas las variables desde el almacén temporal.

1.15.112 SAVE varexpression, varId, expression

Salva una serie de variables en cinta o disco. El primer parámetro indica el número de fichero a guardar, el segundo parámetro es la variable inicial del rango de variables que se guardará el fichero, y el tercero en número de variables a guardar en ese rango (si es cero, se considera como 256). Cualquier cosa que haya en el almacén temporal que se haya cargado con `RAMSAVE` será eliminado, ya que se usará para almacenar el fichero de forma temporal.

- En el caso del disco, se guardará el fichero con el número indicado de tres dígitos y extensión `.SAV`. P.ej. el fichero a cargar con 16 sería el `016.SAV`. Si el fichero ya existiese, se sobrescribirá.
- En el caso de la cinta, se guardará en ésta. El sistema empezará inmediatamente a guardar, con lo que el autor es el responsable de indicar al usuario que se va a grabar y que tiene que poner la unidad de cassette (o equivalente) a grabar. El proceso puede interrumpirse con la tecla **BREAK**.

El resultado de la operación se puede consultar con la función `SAVERESULT()`.

1.15.113 SAVE varexpression, varId

Es el equivalente a `SAVE varexpression, varId, 0`, es decir, guarda todas las variables desde el segundo parámetro hasta el final.

1.15.114 SAVE varexpression

Es el equivalente a `SAVE varexpression, 0, 0`, es decir, guarda todas las variables

1.15.115 LOAD varexpression

Carga una serie de variables desde cinta o disco. El parámetro indica el número de fichero a cargar, y se cargará el rango de variables que se haya indicado al salvar el fichero. Cualquier cosa que haya en el almacén temporal que se haya cargado con `RAMSAVE` será eliminado, ya que se usará para almacenar el fichero de forma temporal.

- En el caso del disco, se cargará el fichero con el número indicado de tres dígitos y extensión `.SAV`. P.ej. el fichero a cargar con 16 sería el `016.SAV`.
- En el caso de la cinta, se cargará el primer bloque sin cabecera que encuentre, con lo que es necesario que el usuario avance la cinta a la posición correcta. El sistema empezará inmediatamente a cargar, con lo que el autor es el responsable de indicar al usuario que se va a cargar y que tiene que poner la unidad de cassette (o equivalente) a reproducir. El proceso puede interrumpirse con la tecla **BREAK**.

El resultado de la carga se puede consultar con la función `SAVERESULT()`.

1.15.116 SAVERESULT()

Función que devuelve el resultado de la última operación de **SAVE** o **LOAD**. Los valores posibles son los siguientes:

0. Resultado correcto.
1. Error al cargar/guardar en cinta/disco. En caso de cinta, operación interrumpida con **BREAK**.
2. La partida grabada no corresponde al juego actual.
3. El slot seleccionado de la partida grabada no corresponde al que se ha solicitado.
4. Error de chequeo del fichero de partida grabada.

Con cualquier otro valor, el resultado está indefinido.

1.15.117 ISDISK()

Función que devuelve 1 si el intérprete es la versión de Plus3 y 0 en caso contrario.

1.15.118 LASTPOS(ID)

Función que devuelve la última posición accesible del array dado. A todos los efectos, el tamaño del array menos 1.

1.16 Imágenes

El motor soporta un máximo de 256 imágenes, aparte de lo que quepa en el disco o la memoria, y deben estar nombradas con un número de 3 dígitos, que corresponderá al número de imagen que se invocará desde el programa. Por ejemplo, la imagen 0 debería llamarse **000.scr**, la imagen número 1 **001.scr**, y así hasta 255. Las imágenes deben estar en el formato de pantalla de Spectrum.

Para mostrar imágenes, el compilador comprime los ficheros en formato SCR de la pantalla de Spectrum en ficheros comprimidos con extensión **csc**. El proceso de compresión puede ser largo, así que el compilador detecta si por cada fichero **scr**, existe un fichero equivalente con extensión **csc** más reciente en el directorio de imágenes. Si es el caso, entonces se salta este fichero, y si no, lo vuelve a comprimir, generando de nuevo el fichero **csc**.

Se puede configurar el número de líneas horizontales de la imagen a mostrar usando el parámetro correspondiente con el compilador para reducir aún más el tamaño. Además, si detecta que la mitad derecha de la misma está espejada con la izquierda, descarta ésta para reducir aún mas el tamaño, aunque podemos forzar este comportamiento con cualquier imagen. Para ello hay que crear un fichero llamado **images.json** dentro del directorio de imágenes.

La estructura del fichero **images.json** es la siguiente:

```
[
  {
    "id": 0,
    "num_lines": 192,
    "force_mirror": false
  },
  {
```

```

    "id": 1,
    "num_lines": 64,
    "force_mirror": true
  }
]

```

Es un listado de registros con los siguientes campos:

- **id** : Es el número de imagen correspondiente a esta entrada. En el ejemplo, "id":0 corresponde a la imagen 000.scr e "id":1 corresponde a la imagen 001.scr.
- **num_lines** : Indica el número de líneas a incluir en el fichero comprimido. El mínimo es 1 y el máximo (pantalla completa) es 192.
- **force_mirror**: Con valor **true**, forzamos a que la imagen se considere simétrica. Esto hará que se descarte la mitad derecha de la imagen y que cuando se descomprima en memoria, se dibuje en su lugar la mitad izquierda reflejada. En otro caso, este campo debe tener el valor **false**.

Debemos indicar en el JSON tantos registros como imágenes queramos definir el comportamiento. Con aquellas imágenes que no estén definidas en el fichero **images.json**, se tratarán con el comportamiento normal, el cual es usar el número de líneas definidas con el parámetro **-il** o 192 por defecto, y usar el espejado sólo en aquellas imágenes detectadas como simétricas.

Hay dos comandos necesarios para mostrar una imagen, el comando **PICTURE n** cargará en un 'buffer' la imagen número n. Es decir, si hacemos **PICTURE 1**, cargará el fichero **001.CSC** en el 'buffer'. Cualquier operación con una imagen requiere su carga previa en dicho 'buffer'. Esto es útil para controlar cuándo se debe cargar la imagen, ya que supondrá espera para la carga desde el disco, además de cierto tiempo para realizar la descompresión de la imagen tanto en cinta como en disco. Por eso es recomendable realizar este comando en un momento adecuado, por ejemplo hacerlo al iniciar un capítulo. Si intentamos cargar una imagen que no exista, recibiremos un error 1, y si se intenta cargar una imagen cuyo fichero no existe en disco, se generará el error de disco 23.

Para mostrar una imagen cargada en el buffer, usamos **DISPLAY n**, donde n tiene que ser cero para ejecutarse. La imagen que se mostrará será la última cargada en el buffer (si existe). La imagen comienza a pintarse desde la esquina superior izquierda de la pantalla y se dibujan tantas líneas como las indicadas al comprimir el fichero y se sobrescribe todo lo que hubiese en pantalla hasta el momento.

También existe un método alternativo para mostrar imágenes, que es el comando **BLIT**, el cual nos permite copiar un fragmento en lugar de una imagen completa en pantalla, además de poder indicar su posición en la misma. El formato del comando es **BLIT xo, yo, ancho, alto AT xd, yd**, y tienes más detalles sobre ella en su [sección](#) correspondiente. **BLIT** es más versátil que **DISPLAY**, pero también es más lento, y también requiere que haya una imagen cargada en el 'buffer' de imagen con **PICTURE** previamente y sobrescribirá cualquier cosa que hubiese en pantalla en la posición donde se dibuje.

1.17 Efectos de sonido

Añadir efectos de sonido con el beeper es muy sencillo. Para ello tenemos que crear un banco de efectos con la utilidad **BeepFx**. Debemos exportar el fichero de efectos como un fichero en

ensamblador usando la opción **File->Compile** del menú superior. En la ventana flotante que aparece, debemos asegurarnos de que tenemos seleccionado **Assembly** e **Include Player Code**, el resto de opciones son irrelevantes, ya que el compilador modificará el fichero fuente para incrustarlo en el intérprete.

Para invocar un efecto, usamos el comando **SFX n**, siendo **n** el número del efecto a reproducir. Si se llama a este comando sin que exista un fichero de efectos cargado, será ignorado y seguirá la ejecución.

No está contemplado llamar a un número de efecto que no exista en el fichero incluido, así que es responsabilidad del autor tener cuidado de esta circunstancia, ya que el comportamiento en ese caso es impredecible.

1.18 Melodías Vortex Tracker

El motor **CYD** permite reproducir módulos de música creados con **Vortex Tracker** en formato **PT3**. Su funcionamiento replica el mecanismo de carga de imágenes, es decir, los módulos deben nombrarse con tres dígitos que representan el número de pista que el intérprete cargará con el comando **TRACK**. Por ejemplo, si el intérprete encuentra el comando **TRACK 3**, entonces buscará la pista del fichero **003.PT3** y cargará en memoria el módulo para su reproducción. Y de la misma manera, el máximo número de módulos que se pueden cargar son 256 (de 0 a 255) y no se permite que el módulo sea de más de 16 Kilobytes para cinta y 8 Kilobytes en caso de usar Plus3.

Una vez cargado un módulo, se podrá reproducir con el comando **PLAY**. Como se indica en la referencia de comandos, si el parámetro es distinto de cero, se reproducirá el módulo; y si es igual a cero, se detendrá la reproducción.

Cuando se llegue al final del módulo, la reproducción se detendrá automáticamente, pero podemos cambiar este comportamiento con el comando **LOOP**. De nuevo, según se indica en la referencia, si el parámetro es igual a cero, al llegar al final se detendrá la reproducción, como se ha indicado antes. Pero si el parámetro es distinto de cero, el módulo volverá a reproducirse desde el principio.

1.19 Melodías WyzTracker

CYD también permite reproducir módulos de música creados con **WyzTracker v2.0**. El funcionamiento es similar al mecanismo de Melodías de **Vortex Tracker** y las imágenes; los nombres de los ficheros de los módulos tienen que ser números de 0 a 255 y con tres dígitos, de tal manera que con **TRACK 3**, cargaríamos el módulo **003.mus** correspondiente.

Una vez cargado un módulo, se podrá reproducir con el comando **PLAY**. Como se indica en la referencia de comandos, si el parámetro es distinto de cero, se reproducirá el módulo; y si es igual a cero, se detendrá la reproducción.

A diferencia de con los módulos de **Vortex Tracker**, no se soporta el comando **LOOP** de forma completa, ya que son los propios módulos los que definen si éstos se reproducirán en bucle o no.

Debido a las peculiaridades del formato de **WyzTracker**, hay que tener en cuenta una serie de consideraciones que afectarán al uso del motor con este reproductor de música:

- Todos los módulos deben compartir los mismos instrumentos. El fichero de instrumentos se debe llamar `instruments.asm` y debe depositarse junto a los ficheros de módulos en el directorio `TRACKS`.
 - *Se reservará la totalidad del banco 1__ de la memoria del Spectrum, en el cual se almacenará el código del reproductor, los instrumentos y las melodías. Esto supone que se dispondrán de 16 Kb menos para la versión de cinta, y 8 Kb menos para la versión de Plus3.
 - Las diferentes melodías serán comprimidas para ahorrar espacio y cada vez que se cargue una de ellas, será descomprimida en el espacio restante del banco 1. El compilador generará un error si alguna de las melodías incluidas no cabe en ese espacio restante.
 - No se soportan efectos de sonido.
-

1.20 Cómo generar una aventura

Lo primero es generar un guion de la aventura mediante cualquier editor de textos empleando la sintaxis arriba descrita. Es MUY recomendable hacer el guion de la misma antes de ponerse a programar la lógica ya que conviene tener el texto perfilado antes para tener una comprensión adecuada (más detalles más adelante).

Es importante que la codificación del fichero sea UTF-8, pero hay que tener en cuenta que caracteres por encima de 128 no se imprimirán bien y sólo se admiten los caracteres propios del castellano, indicados en la sección [Juego de Caracteres](#), que serán convertidos a los códigos allí indicados.

Una vez tenemos la aventura, usamos el compilador `CYDC` para generar un fichero `DSK` o `TAP`. EL compilador busca las mejores abreviaturas para comprimir el texto lo máximo posible. El proceso puede ser muy largo dependiendo del tamaño de la aventura. Por eso es importante tener la aventura perfilada antes, para realizar este proceso al principio. La compilación la realizaremos con el parámetro `-T` de tal manera que con `-T abreviaturas.json`, por ejemplo, exportaremos las abreviaturas encontradas al fichero `abreviaturas.json`.

A partir de este momento, si ejecutamos el compilador con el parámetro `-t abreviaturas.json`, éste no realizará la búsqueda de abreviaturas y usará las que ya habíamos encontrado antes, con lo que la compilación será casi instantánea. Cuando ya consideremos que la aventura está terminada, podremos volver a realizar una nueva búsqueda de abreviaturas para intentar conseguir algo más de compresión.

Para añadir efectos de sonido, imágenes y melodías, consulta las secciones correspondientes.

El proceso es bastante simple, pero tiene algunos pasos dependientes, con lo que se recomienda usar ficheros `BAT` (Windows) o guiones de shell (Linux, Unix) o la utilidad `Make` (o similar) para acelerar el desarrollo.

Para facilitar las cosas, se incluye un programa llamado `make_adventure.py` que realizará todas estas tareas por nosotros, además de los correspondientes scripts `make_adv.cmd` para Windows y `make_adv.sh` para UNIX para ejecutarlo.

El programa `make_adventure.py` buscará y comprimirá automáticamente los ficheros `SCR` que se atengan al formato de nombre establecido (número de 0 a 255 con 3 dígitos) dentro del directorio `.\IMAGES`. Lo mismo hará con los módulos que haya dentro del directorio `.\TRACKS` que cumplan el formato de nombre. Luego compilará el fichero `test.txt` y generará el fichero `tokens.json` con

las abreviaturas, si no existiese previamente. Si se desea que se vuelva a generar el fichero de abreviaturas (es recomendable hacerlo cuando se nos esté agotando la memoria o estemos finalizando la aventura), simplemente borrándolo hará que el script indique al compilador lo genere de nuevo. Además buscará de forma automática si existe un fichero de efectos de sonido llamado **SFX.ASM** que debe generarse con **BeepFX**, y si existiese un fichero JSON con el juego de caracteres llamado **charset.json**, también lo utilizará.

También admite la selección de idioma para los mensajes de salida. Puedes especificar un idioma con el parámetro `--lang {en,es}` o mediante la variable de entorno `CYD_LANG`.

Este programa necesita los directorios `dist` y `tools` con su contenido para realizar el proceso. Ahora se detallan las peculiaridades en cada sistema operativo:

1.20.1 make_adventure.py (helper CLI)

`make_adventure.py` es un wrapper de ayuda sobre `cydc_cli.py` que estandariza rutas del proyecto y automatiza el manejo de tokens.

```
make_adventure.py [opciones] {48k,128k,plus3,mld,mld128} [SJASMPPLUS_PATH]
```

Opciones principales: `--lang {en,es}`: Idioma de salida para los mensajes del helper. `-n, --name NAME`: Nombre base de la aventura (espera `NAME.cyd`, por defecto `test`). `-o, --output-path OUTPUT_PATH`: Directorio para los ficheros de salida. `-img, --images-path, -trk, --tracks-path, -sfx, --sfx-asm-file, -scr, --load-scr-file`. `-tok, --tokens-file`: Ruta de tokens. Si no existe, usa `-T` automáticamente; si existe, usa `-t`. `-chr, --charset-file`: Ruta del JSON de caracteres (se usa si existe). `-il, --image-lines, -l, -L, -s, -S, -trim, -code, --no-strict-colons, -pause, -wyz, -720`.

Nota: tras una compilación `plus3` correcta, limpia automáticamente los ficheros temporales `SCRIPT.DAT`, `DISK` y `CYD.BIN`.

1.20.2 Windows

Como ejemplo se ha incluido el fichero `make_adv.cmd` en la raíz del repositorio, que compilará la aventura de muestra incluida en el fichero `test.cyd`.

Puedes usarlo como base para crear tu propia aventura de forma sencilla. Se puede personalizar el comportamiento modificando en la cabecera del script algunas variables:

```
REM ---- Configuration variables -----

REM Name of the game
SET GAME=test
REM This name will be used as:
REM   - The file to compile will be test.cyd with this example
REM   - The name of the TAP file or +3 disk image

REM Target for the compiler (48k, 128k for TAP, plus3 for DSK, mld o mld128 para MLD)
SET TARGET=48k

REM Number of lines used on SCR files at compressing
SET IMGLINES=192
```

```

REM Loading screen
SET LOAD_SCR="LOAD.scr"

REM Parameters for compiler
SET CYDC_EXTRA_PARAMS=

REM Run emulator after compilation (none/internal/default)
REM     -None: Do nothing
REM     -internal: run the compiled file with Zesarux (must be on the directory .\tools\zesarux\)
REM     -default: run the compiled file with the program set on Windows to run its extension
SET RUN_EMULATOR=none

REM Backup CYD file after compilation (yes/no) on ./BACKUP directory.
SET BACKUP_CYD=no

REM Maximum number of backup files to keep (0 = unlimited)
SET BACKUP_MAX_FILES=0

REM -----

```

- La variable `GAME` será el nombre del fichero txt que se compilará y el nombre del fichero DSK o TAP resultante.
- La variable `TARGET` es el sistema y formato de salida, con estas posibles opciones: – 48k: Genera un fichero TAP para Spectrum 48K, sin soporte de música AY. – 128k: Genera un fichero TAP para Spectrum 128K. – plus3: Genera un fichero DSK para Spectrum +3, con mayor capacidad y carga dinámica de recursos. – mld: Genera un fichero MLD estricto de 48K (sin bancos) para Dandanator. – mld128: Genera un fichero MLD para Dandanator orientado a Spectrum 128K. Los bancos RAM se usan para la música y para los arrays DIM (que deben ser escribibles); el resto de datos de la aventura (texto/imágenes/bytecode) se lee directamente de los slots del cartucho.
- La variable `IMGLINES` es el número de líneas horizontales de los ficheros de imagen que se comprimirán. Por defecto es 192 (la pantalla completa del Spectrum)
- La variable `LOAD_SCR` es la ruta a un fichero de tipo SCR (pantalla de Spectrum) con la pantalla que se usará durante la carga.
- La variable `CYDC_EXTRA_PARAMS` se usa para añadir parámetros extra en la llamada al compilador `cydc`.
- La variable `RUN_EMULATOR` indica si queremos que se ejecute el programa compilado bajo un emulador con los siguientes valores posibles: – none: Si no queremos que haga esto. – internal: Ejecuta el fichero compilado bajo ZEsarUX en `.\tools\ZEsarUX_win-11.0\`. – default: Si la extensión del fichero está asociada bajo Windows con otro emulador, lo ejecutará con éste.
- La variable `BACKUP_CYD` con el valor `yes` hace una copia de seguridad del fichero actual dentro del directorio `.\BACKUP`. Cada copia añade al nombre del fichero la fecha en la cual se creó.
- La variable `BACKUP_MAX_FILES` limita cuántas copias se conservan por juego (0 significa ilimitadas).

El guión producirá un fichero DSK o TAP (dependiendo del formato seleccionado en **TARGET**) que podrás ejecutar con tu emulador favorito. Pero si deseas acelerar más el trabajo, si te descargas [Zesarux](#) y lo instalas en la carpeta `.\tools\ZesarUX_win-11.0`, tras la compilación se ejecutará automáticamente con las opciones adecuadas.

1.20.3 Compilador GUI (make_adventure_gui v1.0.0)

Para aquellos que prefieren una interfaz gráfica en lugar de editar scripts o líneas de comandos, la herramienta **make_adventure_gui** proporciona una GUI multiplataforma para compilar aventuras Choose Your Destiny.

Características: - Soporte multiplataforma (Windows, Linux, macOS con Python 3.11+) - Python integrado en Windows (sin necesidad de instalar Python por separado) - 26 opciones configurables incluyendo objetivos de compilación, rutas y acciones posteriores a la compilación - Persistencia de configuración (recordada entre sesiones) - Soporte completo de internacionalización (Inglés y Español) con cambio de idioma en tiempo real mediante lista desplegable - Visualización en tiempo real del resultado de la compilación - Ocultar ventana de consola en Windows para un inicio limpio

Lanzando la GUI:

Windows:

```
make_adventure_gui.cmd
```

Linux/macOS:

```
./make_adventure_gui.sh
```

Selección de Idioma:

La GUI admite idiomas Español e Inglés con detección automática de tu configuración regional. Puedes cambiar el idioma en cualquier momento usando la lista desplegable **Idioma** en la cabecera - todo el texto de la interfaz se actualizará inmediatamente. La preferencia de idioma se guarda y se restaura de forma automática.

También puedes pre-establecer el idioma con la variable de entorno `CYD_LANG` antes de lanzar la GUI:

```
REM Windows
set CYD_LANG=es
make_adventure_gui.cmd
```

```
# Linux/macOS
export CYD_LANG=es
./make_adventure_gui.sh
```

Opciones Configurables:

Categoría	Opciones
Proyecto	Nombre del juego, Objetivo (48k/128k/plus3/mld/mld128)

Categoría	Opciones
Rutas	Directorio de salida, Ruta de imágenes, Ruta de pistas, Archivo SFX, Pantalla de carga, Archivo de tokens, Conjunto de caracteres, Ejecutable de SjASMPlus
Compilador	Líneas de visualización de imagen, Límites de abreviaturas de texto, Límite de superconjunto, Modo detallado, Dividir textos entre bancos, Trimizar código intérprete, Mostrar bytecode, Compatibilidad de modo colon, Pausa tras carga, Soporte WyzTracker, Disco 720KB
Post-Compilación	Ejecutar emulador después de compilar, Respaldar archivo CYD
Apariencia	Tamaño de fuente, Familia/tamaño/colores de fuente de registro

Todos los archivos compilados se colocan en el directorio de salida especificado, y la GUI mostrará mensajes de compilación detallados para depuración si es necesario.

1.20.4 Linux, BSDs

Como ejemplo se ha incluido el fichero `make_adv.sh` en la raíz del repositorio, que compilará la aventura de muestra incluida en el fichero `test.cyd`.

Puedes usarlo como base para crear tu propia aventura de forma sencilla. Se puede personalizar el comportamiento modificando en la cabecera del script algunas variables:

```
# ---- Configuration variables -----
# Name of the game
GAME="test"
# This name will be used as:
#   - The file to compile will be test.cyd with this example
#   - The name of the TAP file or +3 disk image
#
# Target for the compiler (48k, 128k for TAP, plus3 para DSK, mld o mld128 para MLD)
TARGET="48k"
#
# Number of lines used on SCR files at compressing
IMGLINES="192"
#
# Loading screen
LOAD_SCR="./IMAGES/LOAD.scr"
#
# Parameters for compiler
CYDC_EXTRA_PARAMS=
```

```
# Run emulator after successful compilation (none/internal/custom)
RUN_EMULATOR="none"

# Custom emulator command (used when RUN_EMULATOR=custom)
CUSTOM_EMULATOR_CMD="fuse \${OUTPUT_FILE}"

# Path to ZEsarUX (used when RUN_EMULATOR=internal)
ZESARUX_PATH="./tools/ZEsarUX_linux/zesarux"

# Backup CYD file after compilation (yes/no)
BACKUP_CYD="no"

# Maximum number of backup files to keep (0 = unlimited)
BACKUP_MAX_FILES=0

# -----
```

- La variable `GAME` será el nombre del fichero txt que se compilará y el nombre del fichero DSK o TAP resultante.
- La variable `TARGET` es el sistema y formato de salida, con estas posibles opciones: – 48k: Genera un fichero TAP para Spectrum 48K, sin soporte de música AY. – 128k: Genera un fichero TAP para Spectrum 128K. – plus3: Genera un fichero DSK para Spectrum +3, con mayor capacidad y carga dinámica de recursos. – mld: Genera un fichero MLD estricto de 48K (sin bancos) para Dandanator. – mld128: Genera un fichero MLD para Dandanator orientado a Spectrum 128K. Los bancos RAM se usan para la música y para los arrays `DIM` (que deben ser escribibles); el resto de datos de la aventura (texto/imágenes/bytecode) se lee directamente de los slots del cartucho.
- La variable `IMGLINES` es el número de líneas horizontales de los ficheros de imagen que se comprimirán. Por defecto es 192 (la pantalla completa del Spectrum)
- La variable `LOAD_SCR` es la ruta a un fichero de tipo SCR (pantalla de Spectrum) con la pantalla que se usará durante la carga.
- `RUN_EMULATOR` admite `none`, `internal` (ZEsarUX) o `custom`.
- `CUSTOM_EMULATOR_CMD` se usa cuando `RUN_EMULATOR=custom`.
- `ZESARUX_PATH` define la ruta del ejecutable para el modo `internal`.
- `BACKUP_CYD` y `BACKUP_MAX_FILES` controlan la generación y rotación de copias de seguridad.

1.21 Ejemplos

Dispones de una serie de ejemplos para comprobar las capacidades del motor y aprender de ellos. Cada ejemplo está diseñado para ilustrar características específicas del lenguaje CYD, desde conceptos básicos hasta técnicas avanzadas.

1.21.1 Ejemplos Básicos

1.21.1.1 `examples\test` - Introducción al motor Nivel: Principiante | Requiere: ZX Spectrum 128K

Este es el ejemplo perfecto para empezar. Demuestra los conceptos fundamentales del motor: - **Sistema de opciones y navegación:** Implementa un menú básico con `OPTION` y `CHOOSE`, mostrando

cómo crear ramificaciones en la historia. - **Variables y operaciones:** Usa variables para llevar la cuenta de objetos (“gamusinos”) y realizar operaciones aritméticas básicas. - **Música y sonido:** Incluye una canción de ejemplo en el directorio TRACKS y demuestra cómo reproducir música con PLAY y LOOP. - **Imágenes:** Carga y muestra imágenes SCR desde el directorio IMAGES usando PICTURE y DISPLAY. - **Control de flujo:** Ilustra el uso de GOTO, LABEL, WAITKEY y limpieza de pantalla con CLEAR.

Este ejemplo es una versión ampliada del código mostrado en la sección de Sintaxis del manual.

1.21.1.2 examples\multicolumn_menu - Menús en múltiples columnas Nivel: Principiante

Demuestra cómo organizar opciones en múltiples columnas para crear menús más compactos y visualmente atractivos: - **Configuración de menú:** Usa MENUCONFIG 1,2 para definir un menú de 1 fila y 2 columnas. - **Distribución visual:** Muestra cómo colocar opciones lado a lado en lugar de verticalmente. - **Casos de uso:** Ideal para juegos con muchas opciones, selección de personajes, inventarios, etc.

1.21.1.3 examples\guess_the_number - Juego de adivinanzas Nivel: Principiante-Intermedio

Un juego completo de “adivina el número” que ilustra: - **Números aleatorios:** Genera un número secreto usando RANDOM(1, 32). - **Menú de 6 columnas:** Usa MENUCONFIG 1,6 para crear un menú compacto con 32 opciones (números del 1 al 32). - **Lógica de juego:** Compara el intento del jugador con el número secreto y da pistas (“más alto”/“más bajo”). - **Gestión de variables:** Demuestra cómo usar variables para almacenar el número secreto y los intentos del jugador.

Este ejemplo es excelente para aprender a crear juegos interactivos simples.

1.21.1.4 examples\input_test - Entrada de teclado y arrays Nivel: Intermedio

Ejemplo técnico que muestra características avanzadas de manejo de datos: - **Lectura de teclado:** Captura de entrada del usuario usando INKEY() para leer teclas pulsadas. - **Simulación de arrays:** Usa indirección con corchetes [ptr] para crear y manipular arrays de caracteres. - **Gestión de cadenas:** Implementa funciones para imprimir y editar cadenas de texto de hasta 16 caracteres. - **Subrutinas:** Demuestra el uso de GOSUB y RETURN para crear funciones reutilizables (inputStr y printStr). - **Edición interactiva:** Permite borrar caracteres con DELETE y muestra un cursor parpadeante.

Este ejemplo es fundamental para entender cómo trabajar con datos más complejos y crear interfaces de entrada personalizadas.

1.21.1.5 examples\math_library - Aritmética de 16/32 bits Nivel: Intermedio

Muestra el uso de la librería lib/math16_32.cyd para trabajar con números que se salen de 8 (y de 16) bits: - **Inclusión de librerías:** Incorpora la librería con INCLUDE al principio del programa. - **Multiplicación sin desbordar:** Usa mul1632 (C(32) = A(16) * B(16)) para calcular una puntuación. - **Suma de 32 bits e impresión:** Combina add32 y print32 para mostrar el resultado en decimal. - **Carga de literales anchos:** Emplea la asignación múltiple SET reg T0 {b0, b1, ...}.

1.21.1.6 examples\strings_library - Cadenas de texto Nivel: Intermedio

Muestra el uso de la librería `lib/strings.cyd` para entrada y manejo de cadenas: - **Entrada por teclado:** Lee un nombre con `strInput` (cursor, ENTER, DELETE). - **Impresión y longitud:** Muestra la cadena con `strPrint` y su longitud con `strLen`. - **Buffer configurable:** El autor elige dónde y de qué tamaño es la cadena (`stBase`, `stLen`).

1.21.1.7 `examples\windows` - Ventanas múltiples Nivel: Principiante-Intermedio

Ilustra el sistema de ventanas para dividir la pantalla en diferentes áreas: - **Definición de ventanas:** Crea 4 ventanas usando `WINDOW` y `MARGINS`. - **Áreas independientes:** Cada ventana tiene su propio color de texto (`INK`) y se puede limpiar independientemente. - **Navegación entre ventanas:** Demuestra cómo cambiar entre ventanas para imprimir texto en diferentes áreas. - **Casos de uso:** Útil para crear interfaces tipo SCUMM, separar descripción de acciones, mostrar estadísticas permanentes, etc.

1.21.2 Ejemplos de Nivel Intermedio

1.21.2.1 `examples\ETPA_ejemplo` - Libro “Elige Tu Propia Aventura” Nivel: Intermedio | Requiere: Imágenes en `IMAGES`

Implementación completa de los primeros párrafos de un libro “Elige Tu Propia Aventura” (ETPA): - **Narrativa ramificada:** Demuestra cómo crear una historia con múltiples caminos y finales usando `OPTION` y `GOTO`. - **Variables de configuración:** Usa variables para controlar modos de visualización (solo texto vs. con imágenes, posición de imágenes). - **Sistema de color:** Diferencia entre texto narrativo y opciones usando diferentes colores de tinta. - **Gestión de imágenes:** Carga imágenes `SCR` para ilustrar escenas clave de la aventura. - **Estructura modular:** Organiza el código con etiquetas para cada párrafo/sección (`LABEL s1`, `LABEL s2`, etc.). - **Subrutinas de pantalla:** Usa `GOSUB Pantalla` para gestionar diferentes modos de presentación.

Este ejemplo es ideal para crear aventuras conversacionales o librojuegos interactivos.

1.21.2.2 `examples\include_demo` - Organización multi-archivo Nivel: Intermedio

Demuestra el uso del sistema `INCLUDE` para organizar proyectos grandes: - **Separación de código:** El archivo principal `main.cyd` incluye otros archivos con `INCLUDE "archivo.cyd"`. - **Organización modular:** - `variables.cyd`: Declaraciones de variables - `common.cyd`: Subrutinas compartidas - `intro.cyd`: Secuencia de introducción - `chapter1.cyd` y `chapter2.cyd`: Capítulos independientes - **Mantenibilidad:** Facilita el trabajo en equipo y la organización de proyectos complejos. - **Reutilización:** Las subrutinas comunes pueden compartirse entre múltiples capítulos.

Consulta el archivo `examples\include_demo\README.md` para más detalles.

1.21.2.3 `examples\import_demo` - Rutinas nativas (`IMPORT` / `CALL`) Nivel: Avanzado

Muestra cómo llamar a una rutina Z80 desde un guion con `IMPORT` / `CALL`: - **Código nativo:** un pequeño `peek.asm` lee un byte de una dirección de memoria arbitraria, algo que la máquina virtual no puede hacer por sí sola. - **ABI:** la rutina entra con `DE = FLAGS` e intercambia datos a través de variables. - **Avanzado, bajo tu responsabilidad:** consulta la sección “Rutinas nativas (`IMPORT` / `CALL`)”.

Consulta el archivo `examples\import_demo\README.md` para más detalles.

1.21.2.4 `examples\inline_asm` - Ensamblador inline (ASM / EXPORTS / USES / CYD_CALL) Nivel: Avanzado

Escribe rutinas Z80 nativas **dentro del propio guion** con ASM ... ENDASM y las combina en una pequeña “librería”:

- **ASM / EXPORTS:** un bloque con dos entradas que comparten código, sin fichero aparte.
- **Broker de arrays:** las rutinas leen un array DIM del guion por nombre (ARR_stats) con CYD_ARR_MAP, funcionando también si el array está en un banco paginado.
- **USES / CYD_CALL:** una rutina llama a otras de otro bloque (y otro banco) por el trampolín residente.
- **Código muerto:** las rutinas que nadie alcanza se descartan solas.

Consulta el archivo `examples\inline_asm\README.md` para más detalles.

1.21.3 Ejemplos Avanzados de Gráficos

1.21.3.1 `examples\blit` - Introducción a BLIT Nivel: Intermedio | **Requiere:** Imágenes en IMAGES

Introducción al comando BLIT para copiar sprites y gráficos: - **Copia básica de gráficos:** Usa BLIT x, y, width, height AT col, row para copiar secciones de una imagen. - **Loop de renderizado:** Combina BLIT con bucles WHILE para rellenar la pantalla con un patrón. - **Concepto fundamental:** Base para entender animaciones y gráficos más complejos.

1.21.3.2 `examples\blit_island` - Gráficos avanzados Nivel: Intermedio-Avanzado

Ejemplo más sofisticado del uso de BLIT: - **Composición de escenas:** Construye escenas complejas combinando múltiples sprites. - **Optimización:** Demuestra técnicas para dibujar eficientemente. - **Sprites múltiples:** Gestión de varios elementos gráficos en pantalla.

1.21.3.3 `examples\Rocky_Horror_Show` - Animación de personajes Nivel: Avanzado

Implementa una pantalla de presentación animada: - **Animación frame-by-frame:** Usa BLIT para crear animaciones cambiando los fotogramas. - **Timing preciso:** Combina WAIT con BLIT para controlar la velocidad de animación. - **Sprites grandes:** Manejo de personajes de mayor tamaño en pantalla.

1.21.3.4 `examples\CYD_presents` - Efectos visuales complejos Nivel: Avanzado

Ejemplo de cómo hacer efectos visuales: - **Efectos de presentación:** Implementa animaciones tipo “demo scene” del Spectrum. - **Sincronización:** Coordina múltiples elementos visuales con música. - **Técnicas avanzadas:** Combina BLIT, WAIT, PICTURE y efectos de color.

La serie `blit` → `blit_island` → `Rocky_Horror_Show` → `CYD_presents` forma una progresión natural para dominar los gráficos en CYD.

1.21.3.5 `examples\Golden_Axe_select_character` - Selección dinámica con colores Nivel: Avanzado | **Requiere:** Imágenes en IMAGES

Recrea una pantalla de selección de personajes tipo Golden Axe: - **Detección de cambio de opción:** Usa `CHOOSE IF CHANGED THEN GOSUB` para ejecutar código cuando el jugador mueve el

cursor. - **Efectos de color dinámicos:** Usa `FILLATTR` para cambiar los atributos de color de áreas de la pantalla en tiempo real. - **Resaltado de selección:** Ilumina el personaje seleccionado con un efecto de parpadeo usando bucles. - **Efectos de fade out:** Implementa un efecto de desvanecimiento dividiendo la pantalla en 4 secciones con `FADEOUT`. - **Información contextual:** Muestra texto descriptivo que cambia según el personaje seleccionado. - **Funciones auxiliares:** `OPTIONSEL()` devuelve el índice de la opción actualmente seleccionada.

Este ejemplo es perfecto para crear menús de selección visualmente atractivos e interactivos.

1.21.4 Ejemplos de Proyectos Completos

1.21.4.1 `examples\SCUMM_16` - Interfaz tipo SCUMM/LucasArts Nivel: Avanzado

Implementación completa de una interfaz de aventura gráfica tipo SCUMM (como Monkey Island o Maniac Mansion): - **Ventanas múltiples:** Divide la pantalla en área de gráficos, área de texto y área de verbos. - **Menú de verbos:** Implementa un menú de 9 verbos (3x3) en la parte inferior de la pantalla. - **Sistema de dos niveles:** Primer nivel para seleccionar verbo, segundo nivel para seleccionar objeto/dirección. - **Visualización de sprites:** Usa `BLIT` para copiar la imagen de localidad y los verbos. - **Gestión de objetos:** Imprime una lista de objetos visibles en la localidad. - **Charset personalizado:** Carga un juego de caracteres de 4x8 para texto más compacto usando `CHARSET 1`. - **Retroalimentación visual:** Cambia el color del verbo seleccionado con `FILLATTR`. - **Organización modular:** Usa subrutinas para imprimir objetos, direcciones y gestionar la selección.

Este ejemplo es una plantilla excelente para crear aventuras gráficas de tipo point-and-click.

1.21.4.2 `examples\Delerict` - Aventura completa con motor propio Nivel: Avanzado-Experto | **Requiere:** Imágenes en `IMAGES`

Esqueleto de una aventura espacial completa con todos los sistemas necesarios: - **Sistema completo de objetos:** - Propiedades de objetos (se puede llevar, llevar puesto, abrir, encender, etc.) - Máscara de bits para flags de objetos usando constantes binarias (`0b00000001`) - Gestión de localización de objetos (llevado, puesto, en el suelo, etc.) - **Sistema de localidades:** - Descripciones de localizaciones - Salidas en 8 direcciones (N, S, E, W, Arriba, Abajo, Entrar, Salir) - Flags de localidad (puertas abiertas/cerradas, etc.) - **Sistema de comandos:** - 10 verbos implementados (Examinar, Usar, Coger, Dejar, Ponerse, Quitarse, Encender, Apagar, Abrir, Cerrar) - Parser de acciones con objeto directo - Validación de acciones según propiedades de objetos - **Uso extensivo de constantes:** Define constantes con `CONST` para hacer el código más legible. - **Arquitectura escalable:** Diseñado para añadir fácilmente nuevos objetos, localidades y comandos. - **1428 líneas de código:** Demuestra cómo estructurar proyectos grandes.

Este es el ejemplo más complejo y sirve como base para crear aventuras de texto tradicionales o híbridas (texto + gráficos). Estudiar este código es una clase magistral en programación estructurada con CYD.

1.21.5 Cómo usar los ejemplos

Para probarlos sin compilar: Cada directorio incluye un archivo `.tap` precompilado que puedes cargar directamente en tu emulador de ZX Spectrum favorito.

Para compilarlos tú mismo: 1. Copia el contenido de la carpeta del ejemplo al directorio raíz de la distribución de CYD. 2. **Importante:** Si tienes archivos propios, guárdalos antes, ya que se

sobrescribirán. 3. Ejecuta `make_adv.cmd` (Windows) o `./make_adv.sh` (Linux/macOS). 4. El archivo `.tap` resultante estará listo para probar.

Recomendación de estudio: Se recomienda estudiar los ejemplos en este orden para una curva de aprendizaje gradual: 1. `test` → Conceptos básicos 2. `multicolumn_menu` → Menús 3. `guess_the_number` → Lógica de juego 4. `windows` → Interfaz dividida 5. `input_test` → Entrada avanzada 6. `ETPA_ejemplo` → Narrativa ramificada 7. `include_demo` → Organización de código 8. `blit` → `blit_island` → `Rocky_Horror_Show` → `CYD_presents` → Gráficos progresivamente más complejos 9. `Golden_Axe_select_character` → Efectos visuales dinámicos 10. `SCUMM_16` → Interfaz tipo LucasArts 11. `Delerict` → Plantilla para un motor de aventuras

1.22 Juego de caracteres

El motor soporta un juego de 256 caracteres, con 8 píxeles de altura y tamaño variable de ancho de 1 a 8 píxeles. El juego de caracteres por defecto incluido tiene un tamaño 6x8, junto con un juego alternativo de tamaño 4x8 a partir del carácter 144. Éste es el juego de caracteres por defecto, ordenados de izquierda a derecha y de arriba a abajo:

Los caracteres corresponden con el ASCII estándar, excepto los caracteres propios del castellano, que corresponden a las siguientes posiciones para los dos juegos de caracteres:

Carácter	Posición 6x8	Posición 4x8
‘a’	16	144
‘ı’	17	145
‘i’	18	146
‘«’	19	147
‘»’	20	148
‘á’	21	149
‘é’	22	150
‘í’	23	151
‘ó’	24	152
‘ú’	25	153
‘ñ’	26	154
‘Ñ’	27	155
‘ç’	28	156
‘Ç’	29	157
‘ü’	30	158
‘Ü’	31	159

Los caracteres por encima del valor 127 (empezando desde cero) hasta el 143 (ambos incluidos) son especiales. Son utilizados como iconos en las opciones, es decir, en donde aparece una opción cuando se procesa el comando `OPTION`, y como indicadores de espera con un `WAITKEY` o al cambiar de página si el comando `PAGEPAUSE` está activo.

- El carácter 127 es el carácter usado cuando una opción no está seleccionada en un menú. (En rojo en la captura inferior)



Figure 2: Juego de caracteres por defecto

- Los caracteres del 128 al 135 forman el ciclo de animación de una opción seleccionada en un menú. (En verde en la captura inferior)
- Los caracteres del 135 al 143 forman el ciclo de animación del indicador de espera. (En azul en la captura inferior)

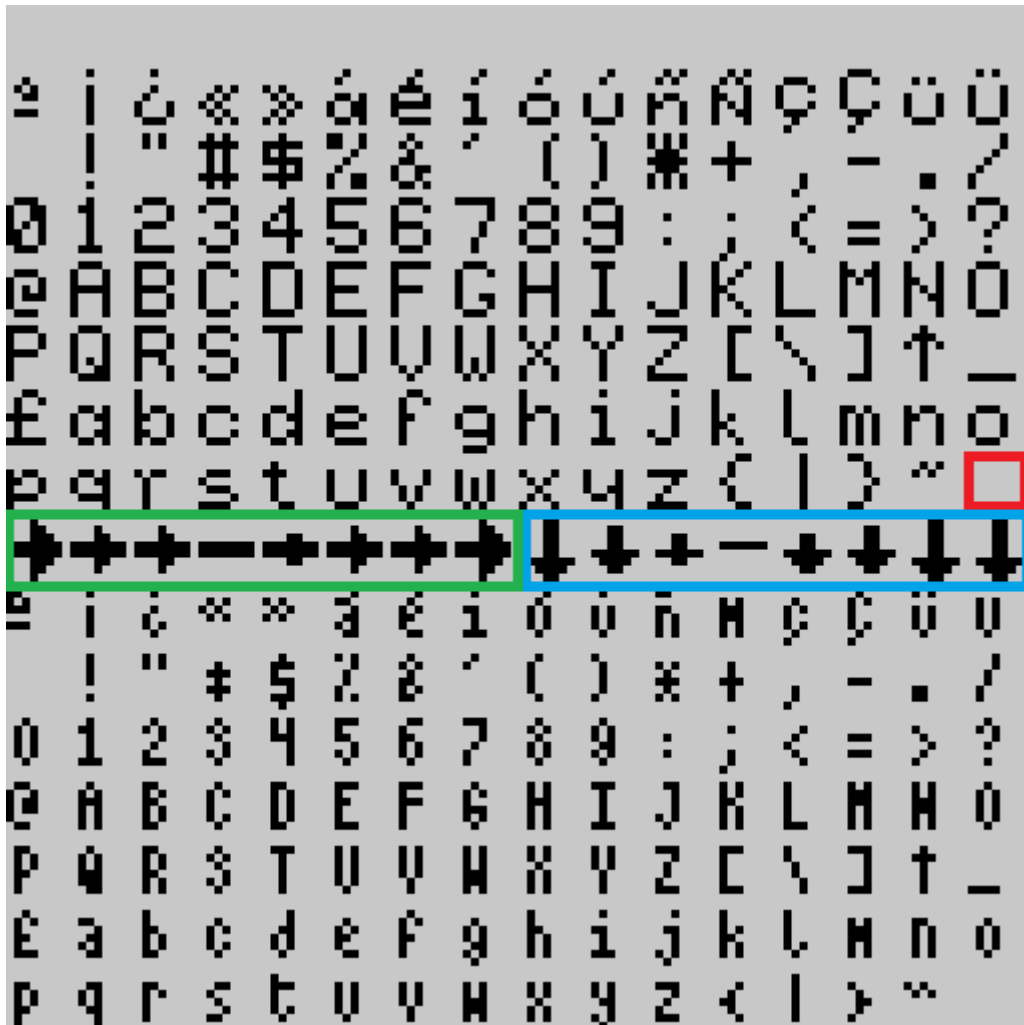


Figure 3: Caracteres especiales

El compilador dispone de dos parámetros, `-c` para importar un juego de caracteres nuevo, y `-C` para exportar el juego de caracteres actualmente empleado, por si puede servir de plantilla o realizar personalizaciones.

Este es el formato de importación/exportación del juego de caracteres:

```
[{"Id": 0, "Character": [255, 128, ...], "Width":8}, {"Id": 1, "Character": [0, 1, ...], "Width":6}, ...]
```

Es un JSON con un array de registros de tres campos:

- *Id*: número del carácter de 0 a 255, que no se puede repetir.
- *Character*: un array de 8 números que corresponde con el valor de los bytes del caracter y, por tanto, no puede haber valores mayores de 255. Cada byte corresponde con los pixels de cada línea del carácter.
- *Width*: un array con el ancho en pixels del carácter (los valores no pueden ser menores que 1 ni mayores que 8). Dado que el tamaño de cada línea del carácter del campo anterior es 8, los

píxeles que sobren por la derecha serán descartados.

Para facilitar la tarea de creación de un juego de caracteres alternativo, se ha incluido la herramienta [CYD Character Set Converter](#) o `cyd_chr_conv`. Esta herramienta permite convertir fuentes en formato `.chr`, `.ch8`, `.ch6` y `.ch4` creadas con ZxPaintbrush en el formato JSON anteriormente mencionado. Además, en el directorio `asest` de la distribución podrás encontrar el fichero `default_charset.chr` con la fuente por defecto para que puedas editarla y personalizarla con dicho programa.

1.23 Códigos de error

La aplicación puede generar errores en tiempo de ejecución. Los errores son de dos tipos, de disco y del motor.

Los errores de motor son, como su nombre indica, los errores propios del motor cuando detecta una situación anómala. Son los siguientes:

- System Error 1: El recurso accedido no existe. Es debido a que se ejecuta `PICTURE` o `TRACK` con un índice que no existe en la aventura, debido a que no se ha cargado la imagen o pista correspondiente al compilar. También sucede cuando se hace un `RETURN` sin hacer un `GOSUB` previo.
- System Error 2: Se han creado demasiadas opciones, se ha superado el límite de opciones posibles.
- System Error 3: No hay opciones disponibles, se ha lanzado un comando `CHOOSE` sin tener antes ninguna `OPTION` declarada.
- System Error 4: El fichero con el módulo de música a cargar es demasiado grande, tiene que ser menor que 16Kib.
- System Error 5: No hay un módulo de música cargado para reproducir.
- System Error 6: Código de instrucción inválido.
- System Error 7: Acceso a posición del array fuera del rango.
- System Error 8: Opción perdida. Al hacer scroll, las opciones declaradas se desplazan hacia arriba, si una de ellas sale por el límite superior de los márgenes, se genera este error.

Los errores de disco son los errores que pudiesen ocasionarse cuando el motor del juego accede al disco, y corresponden con los errores de +3DOS:

- Disk Error 0: Drive not ready
- Disk Error 1: Disk is write protected
- Disk Error 2: Seek fail
- Disk Error 3: CRC data error
- Disk Error 4: No data
- Disk Error 5: Missing address mark
- Disk Error 6: Unrecognised disk format
- Disk Error 7: Unknown disk error
- Disk Error 8: Disk changed whilst +3DOS was using it
- Disk Error 9: Unsuitable media for drive
- Disk Error 20: Bad filename

- Disk Error 21: Bad parameter
- Disk Error 22: Drive not found
- Disk Error 23: File not found
- Disk Error 24: File already exists
- Disk Error 25: End of file
- Disk Error 26: Disk full
- Disk Error 27: Directory full
- Disk Error 28: Read-only file
- Disk Error 29: File number not open (or open with wrong access)
- Disk Error 30: Access denied (file is in use already)
- Disk Error 31: Cannot rename between drives
- Disk Error 32: Extent missing (which should be there)
- Disk Error 33: Uncached (software error)
- Disk Error 34: File too big (trying to read or write past 8 megabytes)
- Disk Error 35: Disk not bootable (boot sector is not acceptable to DOS BOOT)
- Disk Error 36: Drive in use (trying to re-map or remove a drive with files open)

La aparición de estos errores ocurre cuando se accede al disco, al buscar más trozos de texto, imágenes, etc. Si aparece el error 23 (File not found), suele ser que se haya olvidado de incluir algún fichero necesario en el disco. Otros errores ya suponen algún error de la unidad de disco o del propio disco.

1.24 Referencias y agradecimientos

- David Beazley por [PLY](#)
- Einar Saukas por el compresor [ZX0](#) y [ZX7](#).
- Mokona por su versión del [compresor ZX0 para Python](#).
- Sylvain Glaize por la versión del [descompresor ZX0 para Python](#).
- DjMorgul por el buscador de abreviaturas, adaptado de [Daad Reborn Tokenizer](#)
- Shiru por [BeepFx](#).
- Seasip por mkp3fs de [Taptools](#).
- [Sergey.V.Bulba](#) por el reproductor de Vortex Tracker.
- Augusto Ruiz por el [reproductor de WyzTracker](#).
- Al equipo responsable del ensamblador [sjasmplus](#).
- [Tranqui69](#) por el logotipo y su apoyo.
- XimoKom, Javier Fopiani y Fran Kapilla por su inestimable ayuda en las pruebas del motor.
- Pablo Martínez Merino por la ayuda con el testeo en Linux y ejemplos.
- Sergio ThePoPe por meterme el gusanillo del Plus3.
- [El_Mesías](#), [Arnau Jess](#) y [Javi Ortiz](#) por la difusión.
- Al Club de Aventuras AD [CAAD](#) por el apoyo.
- A todos los miembros del [grupo de Telegram de Choose your Destiny](#).

1.25 Licencias

Este paquete de software está sometido a diferentes licencias dependiendo de sus componentes:

- El compilador está bajo licencia [GNU Affero General Public License v3.0](#).
- El intérprete (la parte ejecutable en la máquina objetivo) se encuentra bajo la siguiente licencia:

Choose Your Destiny

Copyright (c) 2025 Sergio Chico (Cronomantic)

Por la presente se concede permiso, libre de cargos, a cualquier persona que obtenga una copia de este

- El aviso de copyright anterior y este aviso de permiso se incluirán en todas las copias o partes sust
- El aviso del copyright anterior y/o uno de los logotipos del proyecto deben estar indicados de forma
- EL SOFTWARE SE PROPORCIONA "COMO ESTÁ", SIN GARANTÍA DE NINGÚN TIPO, EXPRESA O IMPLÍCITA, INCLUYENDO

- [PyZX0](#), está sometido a licencia [BSD-3](#).
- El [reproductor de WyzTracker](#) está bajo licencia [MIT](#)
- [PLY](#) y el [reproductor de WyzTracker](#) no tienen licencias específicas, se asume una licencia similar a BSD o MIT.
- [BeepFx](#) está bajo licencia [WTFPL v.2](#).

En términos sencillos, significa que si usas este motor para hacer un juego, siempre tienes que indicar en la pantalla de carga o dentro del propio juego que se ha realizado con CYD o usar alguno de los logotipos de proyecto (incluidos en el directorio `assets`). También debes hacerlo en la web de descarga si distribuyes el juego online, y en la caja o recipiente del medio si lo distribuyes de forma física. Aparte de la anterior condición, el juego realizado **SIEMPRE** será de tu propiedad y autoría; puedes venderlo o distribuirlo sin necesidad de publicar el código fuente ni recursos artísticos. Sin embargo, la herramienta está bajo licencia AGPL, lo que significa que si la modificas o haces una versión propia, tienes obligación de publicar el código e indicar que está basado en este proyecto.